Banking Data Governance & Data Quality Pipeline using PostgreSQL, Python, PySpark, SQL, DuckDB, OpenMetadata Concepts, and Agile/SDLC Artifacts

---

1. Executive Summary

The **Banking Data Governance & Data Quality Pipeline** is a hands-on technical and governance practice project designed to simulate how banking data moves from an operational source system into governed, validated, analytics-ready data assets. The project was built as a local practice lab for strengthening skills required in Data Governance, Collibra-style governance, Data Quality Engineering, Data Catalog, Data Lineage, SQL, PySpark, Python scripting, Agile/Scrum, SDLC, data modeling, and data warehousing roles.

The project implements an end-to-end banking data pipeline using synthetic data. It begins with a normalized PostgreSQL banking source system populated through a Faker-based data generator. Source data is extracted through a Python ingestion layer into a raw Parquet zone, standardized using PySpark, modeled into curated fact and dimension datasets, validated through both SQL and PySpark data quality frameworks, and then connected to governance artifacts such as a business glossary, data catalog, Critical Data Elements, source-to-target mappings, lineage documentation, data quality rule catalog, and stewardship issue examples.

The completed project demonstrates the following end-to-end flow:

Faker Synthetic Data Generator
→ **PostgreSQL Banking Source System**
→ **Python Ingestion Layer**
→ **Raw Parquet Zone**
→ **PySpark Standardized Processing Layer**
→ **Curated Warehouse Layer**
→ **SQL Data Quality Validation using DuckDB**
→ **PySpark Data Quality Validation**
→ **Governance Artifacts**
→ **OpenMetadata Integration Documentation and Templates**
→ **Agile / SDLC Delivery Evidence**

The technical pipeline was built around a realistic banking domain model. The source system includes 10 PostgreSQL source and control tables covering branches, products, reference codes, customers, accounts, transactions, customer risk scores, data quality rules, stewardship issues, and source-to-target mappings. The generated data includes 500 customers, 990 accounts, 5,000 transactions, 742 customer risk score records, 12 data quality rule records, 8 sample stewardship issue records, and 25 source-to-target mapping records.

The engineering layer includes Python-based extraction from PostgreSQL into load-date-partitioned raw Parquet files. The ingestion process includes table whitelist validation, load-date validation, logging, summary CSV generation, and Parquet read-back row-count verification. This provides a practical ingestion pattern that resembles a controlled raw landing process in a data lake environment.

The PySpark processing layer reads the raw Parquet files, applies standardization, schema handling, type casting, required-column checks, metadata enrichment, string cleanup, and row-count validation. The standardized data is then used to build a curated warehouse layer containing seven analytics-ready models:

Model	***Type***	***Purpose***
dim_customer	Dimension	Customer reporting, segmentation, governance, and DQ validation
dim_account	Dimension	Account ownership, product relationship, and account analytics
dim_branch	Dimension	Branch-level reporting and relationship analysis
dim_product	Dimension	Product ownership and account-product analysis
fact_transaction	Fact	Transaction analysis, reconciliation, fraud review, and financial reporting
fact_customer_risk_score	Fact	Customer risk monitoring and risk trend analysis
fact_daily_account_balance	Fact	Account-day balance summary analysis

The project includes two separate data quality frameworks. The SQL Data Quality framework uses DuckDB to query Parquet files directly and execute SQL validation scripts over the curated and standardized layers. It includes null checks, duplicate logical-key checks, accepted-value checks, referential-integrity checks, source-to-target reconciliation, control-total validation, and exception-record queries. The full SQL validation suite completed with **54 checks, 0 failed checks, and 0 exception records**.

The PySpark Data Quality framework provides a separate scalable validation layer using Spark DataFrame operations, filters, joins, grouping, and aggregations. It performs dataset profiling, column profiling, schema validation, required-column validation, duplicate logical-key validation, accepted-value checks, range checks, referential-integrity checks, source-to-target reconciliation, control-total validation, and bounded exception-record generation. All seven curated datasets passed the PySpark validation process with **0 failed rule counts and 0 exception records**.

The governance layer connects the technical pipeline to enterprise data governance concepts. It includes a governance policy, business glossary, Critical Data Element inventory, data catalog, source-to-target mappings, lineage mappings, data quality rule catalog, and sample stewardship issue records. The project defines 15 Critical Data Elements, including fields such as customer_id, customer_number, email, date_of_birth, account_id, account_number, account_status, current_balance, transaction_id, transaction_reference, transaction_amount, transaction_date, transaction_type, risk_score, and risk_band.

The OpenMetadata portion of the project was intentionally implemented as a governance-platform integration design, not as a full production deployment. OpenMetadata was used as an open-source counterpart for practicing Collibra-style governance concepts such as cataloging, glossary management,

ownership, classifications, lineage, and metadata ingestion design. The project includes OpenMetadata integration documentation, safe metadata ingestion configuration templates, and a local deployment runbook. No OpenMetadata runtime ingestion was executed, and no production OpenMetadata deployment was performed.

The Agile / SDLC layer demonstrates how governance and data quality work would be planned, tracked, validated, and released in an enterprise delivery model. It includes 19 Agile user stories across Governance, Catalog, Lineage, Data Quality, and OpenMetadata epics; a 4-sprint backlog; story points; owners; acceptance criteria; Definition of Done; test evidence; release notes; and a stewardship workflow. This connects the technical and governance deliverables to a realistic delivery lifecycle.

Overall, the project demonstrates a complete local banking governance and data quality practice lab. It is not a production banking platform and does not use real customer data. It is a structured, synthetic, local implementation designed for interview preparation, portfolio demonstration, technical walkthroughs, and hands-on understanding of how data engineering, data quality, metadata management, lineage, cataloging, stewardship, and Agile delivery fit together in a governed data environment.

---

2. Business Problem and Objectives

2.1 Business Problem

Banking organizations depend on trusted data for customer reporting, account analysis, transaction monitoring, risk management, reconciliation, regulatory reporting, fraud review, and operational decision-making. In a real banking environment, data often originates from multiple operational systems and moves through several layers before it becomes available for analytics or reporting.

Without proper governance and data quality controls, several problems can occur:

Problem Area	***Business Risk***
Undefined business terms	Teams may interpret fields such as customer, account, transaction amount, or risk score differently.
Missing catalog and ownership	Users may not know which datasets exist, who owns them, or whether they are approved for use.
Weak source-to-target traceability	Analysts and auditors may not be able to explain where curated reporting fields came from.
Poor data quality controls	Null keys, duplicate records, invalid statuses, orphan records, and incorrect totals can reduce trust in reporting.
Missing lineage	Data consumers may not understand how data moved from source systems into curated outputs.

No stewardship workflow	Failed data quality checks may not have a clear owner, remediation path, or closure process.
Weak SDLC evidence	Governance work may lack user stories, test evidence, release notes, approval criteria, and audit-ready documentation.

This project was created to simulate those challenges in a controlled local environment. Instead of only building a technical ETL pipeline, the project connects data engineering work to governance outcomes: glossary terms, Critical Data Elements, catalog assets, data quality rules, lineage, ownership, stewardship, validation evidence, and Agile delivery artifacts.

The core business problem addressed by the project is:

How can a banking data pipeline be designed, processed, validated, cataloged, governed, and documented so that downstream users can trust curated data assets and understand their meaning, quality, ownership, and lineage?

2.2 Project Objectives

The project objectives were designed around both technical implementation and governance readiness.

2.2.1 Build a Realistic Banking Source System

The first objective was to simulate a normalized banking source system using PostgreSQL and synthetic data. The source model needed to be more realistic than a simple customer-account-transaction example. It needed supporting entities, reference-style records, risk data, quality rule records, stewardship examples, and source-to-target mapping records.

The source model therefore includes:

Source / Control Table	Purpose
branches	Banking branch reference data
products	Banking product reference data
reference_codes	Controlled reference values
customers	Customer master data
accounts	Customer account data
transactions	Banking transaction activity
customer_risk_scores	Customer risk scoring records
data_quality_rules	Data quality rule metadata
stewardship_issues	Sample stewardship issue tracking records

Source / Control Table	Purpose
source_to_target_mappings	Source-to-target mapping metadata

This objective supports source-system understanding, schema design, normalized data modeling, referential integrity, and banking-domain practice.

2.2.2 Build a Python Ingestion Layer

The second objective was to move data from PostgreSQL into a raw analytical landing zone using Python. The ingestion process needed to simulate a controlled data lake landing pattern rather than a one-time export.

The ingestion layer was designed to:

Objective	*Implementation*
Extract approved tables	Python extraction script with table whitelist validation
Preserve load history	Load-date-partitioned Parquet output
Validate output	Parquet read-back row-count comparison
Support operations	Logging and summary CSV generation
Prevent unsafe usage	Invalid table and invalid load-date rejection
Protect configuration	Environment-variable-based PostgreSQL connection handling

This objective supports Python scripting, database connectivity, file-based ingestion, operational logging, validation, and raw zone design.

2.2.3 Build a PySpark Standardized Processing Layer

The third objective was to process raw Parquet data using PySpark and create a standardized data layer. This layer needed to improve structure, consistency, and traceability before curated modeling.

The PySpark standardized layer was designed to:

Objective	*Implementation*
Read raw data	Load-date-partitioned raw Parquet input
Standardize schema	Lowercase snake_case column handling and table-specific schema configuration
Clean data values	String trimming, blank-to-null handling, email lowercasing, and status/code normalization
Enforce expected structure	Required-column checks and type casting

Add traceability	Metadata columns such as source load date and standardized processing timestamp
Validate processing	Raw-to-standardized row-count verification

This objective supports Spark processing, schema handling, data standardization, type normalization, traceability, and controlled transformation design.

2.2.4 Build a Curated Warehouse Layer

The fourth objective was to model the standardized data into analytics-ready curated datasets using fact and dimension design. The curated warehouse layer needed to support reporting, governance, lineage, and downstream quality validation.

The curated layer includes:

Curated Model	***Grain***
dim_customer	One row per customer
dim_account	One row per account
dim_branch	One row per branch
dim_product	One row per product
fact_transaction	One row per transaction
fact_customer_risk_score	One row per customer risk score record
fact_daily_account_balance	One row per account per balance date

This objective supports dimensional modeling, grain definition, logical key design, relationship documentation, curated Parquet output generation, and warehouse-style data architecture.

2.2.5 Implement SQL-Based Data Quality Validation

The fifth objective was to build a SQL validation framework that checks curated warehouse data for quality and reconciliation issues. DuckDB was selected because the curated and standardized datasets are stored as Parquet files, and DuckDB can query Parquet directly through SQL without requiring an additional persistent warehouse load.

The SQL framework validates:

Validation Category	***Purpose***
Null checks	Confirm critical fields and keys are populated
Duplicate logical-key checks	Confirm logical primary keys are unique
Accepted-value checks	Confirm controlled fields contain expected values

Referential-integrity checks	Confirm fact records map to valid dimension records
Source-to-target reconciliation	Confirm records were not lost during transformation
Control-total checks	Confirm financial and count totals remain consistent
Exception-record queries	Return failed records for investigation and stewardship review

This objective supports practical SQL validation, audit-style reconciliation, control totals, exception reporting, and governance evidence generation.

2.2.6 Implement PySpark-Based Data Quality Validation

The sixth objective was to build a separate PySpark validation framework to practice scalable data profiling and quality checks. This framework does not replace SQL validation. It complements it by demonstrating how validation can be performed using Spark DataFrame logic, joins, filters, grouping, and aggregations.

The PySpark DQ framework validates:

Validation Category	***Purpose***
Dataset profiling	Row count, column count, expected/missing/extra column tracking
Column profiling	Null count, non-null count, distinct count
Schema validation	Confirm expected columns exist
Required-column validation	Confirm required columns are present
Duplicate logical-key validation	Identify duplicate logical keys
Accepted-value validation	Validate controlled categorical fields
Range validation	Validate numeric and financial fields
Referential-integrity validation	Use Spark joins to detect orphan relationships
Source-to-target reconciliation	Compare standardized and curated row counts
Control-total validation	Compare financial totals and grouped counts
Exception-record generation	Capture failed records for review

This objective supports scalable DQ validation, profiling, exception handling, Spark joins, aggregations, and evidence generation.

2.2.7 Create Governance Artifacts

The seventh objective was to connect the technical data pipeline to governance practices. The project needed to show how data engineering outputs become governed assets with business meaning, ownership, quality rules, lineage, and stewardship accountability.

The governance artifacts include:

Governance Artifact	*Purpose*
Governance policy	Defines operating rules, ownership, stewardship, quality expectations, lineage expectations, and change control
Business glossary	Defines business terms used across the banking pipeline
Critical Data Elements	Identifies important governed fields and maps them to classifications, owners, stewards, assets, and DQ rules
Data catalog	Documents technical data assets across source, raw, standardized, curated, and DQ layers
Source-to-target mapping	Documents how source fields flow into curated outputs
Lineage mapping	Documents asset-level and field-level lineage
DQ rule catalog	Documents validation categories implemented in SQL and PySpark
Stewardship issues	Provides sample issue records for governance and DQ triage practice

This objective supports business glossary design, cataloging, CDE management, lineage, data ownership, data stewardship, and governance documentation.

2.2.8 Map Governance Concepts to OpenMetadata

The eighth objective was to demonstrate how the governance artifacts could be represented in an open-source governance and catalog platform. OpenMetadata was used as an open-source counterpart for practicing Collibra-style concepts.

This objective was intentionally documentation-driven and configuration-driven. The project includes:

OpenMetadata Component	*Status*
OpenMetadata integration README	Completed
PostgreSQL metadata ingestion template	Completed
Curated layer metadata ingestion template	Completed
Lineage ingestion template	Completed
Local OpenMetadata runbook	Completed
Runtime OpenMetadata installation	Not executed
Runtime metadata ingestion	Not executed
Production OpenMetadata deployment	Not implemented

This objective supports governance platform thinking, metadata ingestion planning, secure configuration patterns, catalog mapping, glossary mapping, ownership, classification, and lineage representation.

2.2.9 Simulate Agile / SDLC Governance Delivery

The ninth objective was to show how governance and data quality work would be delivered in a real Agile/Scrum environment. Governance work is not only technical. It also requires planning, acceptance criteria, testing, review, approvals, release notes, and support workflows.

The Agile / SDLC layer includes:

Agile / SDLC Artifact	***Purpose***
User stories	Capture governance, catalog, lineage, DQ, and OpenMetadata requirements
Sprint backlog	Organize completed work into a 4-sprint delivery model
Acceptance criteria	Define what "done" means for each story
Definition of Done	Define release-readiness criteria
Test evidence	Document validation results from completed phases
Release notes	Summarize delivered capabilities
Stewardship workflow	Define issue lifecycle from detection to closure

This objective supports JIRA-style delivery, Agile planning, Scrum structure, SDLC evidence, audit readiness, release management, and governance operations.

2.3 Overall Project Success Criteria

The project is considered successful because it demonstrates the complete lifecycle of a governed banking data pipeline:

Success Area	***Completed Outcome***
Source simulation	Synthetic banking data generated and loaded into PostgreSQL
Ingestion	PostgreSQL tables extracted into raw Parquet with validation
Standardization	Raw data standardized using PySpark with schema handling
Curated modeling	Seven fact/dimension-style curated models created
SQL DQ	SQL validation framework implemented using DuckDB
PySpark DQ	Spark-based profiling and DQ validation framework implemented
Governance artifacts	Glossary, CDEs, catalog, lineage, DQ rules, stewardship examples, and policy created

OpenMetadata concepts	Integration documentation, ingestion templates, and runbook created
Agile/SDLC	User stories, sprint backlog, Definition of Done, test evidence, release notes, and stewardship workflow created
Interview readiness	Project connects technical work to governance, quality, lineage, catalog, and SDLC concepts

The completed project provides a practical and explainable foundation for discussing Data Governance, Data Quality Engineering, Collibra-style concepts, OpenMetadata, SQL validation, PySpark validation, data modeling, lineage, cataloging, stewardship, and Agile delivery in interviews and portfolio walkthroughs.

3. Project Scope

3.1 Scope Overview

The Banking Data Governance & Data Quality Pipeline was built as a local, hands-on practice project that simulates how a banking data platform can be engineered, validated, governed, cataloged, and documented. The scope intentionally combines technical data engineering work with governance, metadata, lineage, data quality, and Agile/SDLC delivery artifacts.

The project is not only an ETL pipeline. It is an end-to-end governed data pipeline practice lab that demonstrates how source data moves through raw, standardized, and curated layers, how data quality is validated through SQL and PySpark, and how the resulting data assets are connected to governance concepts such as business glossary, Critical Data Elements, cataloging, lineage, ownership, stewardship, and issue tracking.

The project was completed using synthetic banking data and local development tooling. It does not use real customer data, does not represent a production banking platform, and does not claim production deployment.

3.2 Implemented Components

The following components were implemented as part of the completed project:

Area	Implemented Component	Description
Source system	PostgreSQL banking source model	Normalized banking source schema with 10 source/control tables
Data generation	Faker-based synthetic data generator	Populated PostgreSQL with synthetic banking data
Ingestion	Python ingestion layer	Extracted approved PostgreSQL tables into raw Parquet files

Area	Implemented Component	Description
Raw zone	Local Parquet raw layer	Stored source extracts using load-date partitioned folder structure
Processing	PySpark standardized layer	Standardized raw data, applied schema handling, type casting, and metadata columns
Curated modeling	Curated warehouse layer	Built seven fact/dimension-style curated models
SQL DQ	DuckDB SQL validation framework	Queried Parquet files directly and executed SQL DQ checks
PySpark DQ	Spark-based DQ framework	Performed profiling, validation, reconciliation, and exception-record generation
Governance artifacts	Markdown and YAML governance files	Created glossary, catalog, CDEs, mappings, lineage, DQ rules, policy, and stewardship examples
OpenMetadata preparation	OpenMetadata documentation and templates	Created integration documentation, metadata ingestion templates, and local runbook
Agile / SDLC	Agile delivery artifacts	Created user stories, sprint backlog, Definition of Done, test evidence, release notes, and stewardship workflow

3.3 Documentation-Only and Design-Only Components

Some components were intentionally documentation-only or design-only. This distinction is important for technical accuracy and interview credibility.

Component	Status	Clarification
OpenMetadata integration	Documentation and template design completed	OpenMetadata was used as an open-source counterpart for Collibra-style governance concepts. Runtime installation and ingestion were not executed.
OpenMetadata metadata ingestion	Templates created only	PostgreSQL, curated layer, and lineage ingestion YAML templates were created with secure placeholders. No ingestion job was executed.
OpenMetadata local deployment	Runbook created only	A local deployment and validation runbook was created. The platform was not deployed as part of the completed implementation.
Collibra	Not implemented	The project practices Collibra-style concepts but does not use or claim Collibra implementation.

Component	Status	Clarification
Agile/JIRA	Simulated through files	JIRA-style user stories and sprint backlog were created using Markdown/YAML, not an actual JIRA instance.
Production orchestration	Not implemented	No production scheduler such as Airflow was added to the active implementation scope.
Cloud deployment	Not implemented	The pipeline runs as a local practice lab using local PostgreSQL, local Parquet, and local validation outputs.

3.4 In Scope

The project scope includes the following major workstreams.

Data Engineering Scope

Scope Item	Description
Source schema design	Design of a normalized PostgreSQL banking schema
Synthetic data generation	Faker-based data generation for banking entities
Python ingestion	PostgreSQL-to-Parquet extraction with validation
Raw data landing	Load-date-partitioned raw Parquet output
PySpark standardization	Raw-to-standardized processing with schema handling
Curated modeling	Fact and dimension-style curated datasets
Local output generation	Parquet outputs and CSV summary evidence

Data Quality Scope

Scope Item	Description
SQL validation	SQL checks using DuckDB over Parquet
Null checks	Critical-field and required-key validation
Duplicate checks	Logical-key duplicate detection
Accepted-value checks	Controlled field validation
Referential-integrity checks	Fact-to-dimension relationship validation
Source-to-target reconciliation	Source-like standardized layer compared to curated outputs

Scope Item	Description
Control totals	Financial and count-based reconciliation
Exception records	Bounded failed-record outputs for review
PySpark validation	Spark-based profiling, rule execution, joins, aggregations, and exception output

Governance Scope

Scope Item	Description
Governance policy	Operating rules for governance, ownership, stewardship, lineage, DQ, and change control
Business glossary	Banking business terms mapped to technical context
Critical Data Elements	CDE inventory with classifications, owners, stewards, fields, assets, and DQ linkage
Data catalog	Catalog entries for source, raw, standardized, curated, and DQ result assets
Source-to-target mapping	Field and asset mapping from source structures to curated outputs
Lineage mapping	Asset-level and field-level lineage documentation
Data quality rule catalog	Cataloged DQ categories implemented in SQL and PySpark
Stewardship issues	Sample governance/DQ issue records and remediation context

OpenMetadata Scope

Scope Item	Description
OpenMetadata integration design	Documented how governance artifacts map to OpenMetadata concepts
Metadata ingestion templates	Created safe YAML templates for PostgreSQL, curated assets, and lineage
Local runbook	Documented future local deployment and validation steps
Secure configuration usage	Used environment-variable references and avoided committed secrets

Agile / SDLC Scope

Scope Item	Description
User stories	Governance, Catalog, Lineage, Data Quality, and OpenMetadata stories
Sprint backlog	Four-sprint delivery model
Definition of Done	Criteria for completing governance and DQ work
Test evidence	Validation evidence from completed components
Release notes	Summary of delivered project capabilities
Stewardship workflow	Issue lifecycle from detection through closure

3.5 Out of Scope

The following items were intentionally excluded to keep the project focused and accurate:

Out-of-Scope Item	Reason
Real banking data	The project uses synthetic data only.
Production deployment	This is a local practice lab, not an enterprise production platform.
Collibra implementation	Collibra is closed-source enterprise software; the project practices Collibra-style concepts without claiming Collibra usage.
OpenMetadata runtime ingestion	Templates and runbook were created, but runtime ingestion was not executed.
OpenMetadata production deployment	Deployment was documented but not performed.
Production scheduler	No Airflow or enterprise scheduler was added to active implementation.
Cloud object storage	Local Parquet was used instead of S3, ADLS, GCS, or production object storage.
Dashboarding	No BI dashboard layer was implemented.
Automated stewardship workflow engine	Stewardship workflow was documented; automated workflow execution was not implemented.
CI/CD pipeline	Git commits and SDLC artifacts were created, but no full CI/CD automation pipeline was implemented.
Real JIRA integration	Agile artifacts were simulated using local Markdown/YAML files.

3.6 Scope Control Principle

The project followed a disciplined scope-control principle:

Each layer was designed, implemented, validated, and documented before moving to the next layer.

This helped prevent the project from becoming a loose collection of scripts. Instead, the project evolved in a controlled SDLC-style sequence:

Foundation → Source System → Ingestion → Standardization → Curated Modeling → SQL DQ → PySpark DQ → Governance Artifacts → OpenMetadata Planning → Agile/SDLC Evidence

This scope discipline makes the final project easier to explain in interviews because each component has a clear purpose, boundary, output, and validation result.

---

4. Architecture Overview

4.1 Architecture Summary

The Banking Data Governance & Data Quality Pipeline uses a layered architecture. Each layer has a clear responsibility and produces outputs consumed by the next layer. The architecture begins with synthetic banking data generation and ends with governance, metadata, quality evidence, and Agile/SDLC documentation.

The full architecture is:

Faker Synthetic Data Generator
→ PostgreSQL Banking Source System
→ Python Ingestion Layer
→ Raw Parquet Zone
→ PySpark Standardized Processing Layer
→ Standardized Parquet Zone
→ Curated Warehouse Layer
→ SQL Data Quality Validation using DuckDB
→ PySpark Data Quality Validation
→ Governance Artifacts
→ OpenMetadata Integration Documentation and Templates
→ Agile / SDLC Delivery Evidence

4.2 Architecture Layers

Layer	Main Purpose	Main Output
Data generation layer	Create synthetic banking data	PostgreSQL-ready source records
PostgreSQL source layer	Simulate normalized banking source system	10 source/control tables
Python ingestion layer	Extract source tables to raw data lake format	Raw Parquet files

Layer	Main Purpose	Main Output
Raw Parquet zone	Preserve source-like extracts	Load-date-partitioned raw datasets
PySpark standardized layer	Clean, standardize, type-cast, and add metadata	Standardized Parquet files
Curated warehouse layer	Build analytics-ready fact and dimension models	Seven curated Parquet models
SQL DQ layer	Validate curated data using SQL	SQL DQ result files and summary logs
PySpark DQ layer	Validate curated data using Spark logic	PySpark DQ result files and summary logs
Governance artifact layer	Document business meaning, ownership, CDEs, catalog, lineage, and DQ rules	Markdown/YAML governance files
OpenMetadata integration layer	Map governance artifacts to OpenMetadata concepts	README, ingestion templates, local runbook
Agile / SDLC layer	Simulate enterprise delivery management	User stories, backlog, DoD, test evidence, release notes

4.3 Data Movement Architecture

The data movement architecture follows a source-to-curated pattern.

Step	Input	Process	Output
1	Synthetic data rules	Faker generator inserts records	PostgreSQL source tables
2	PostgreSQL source tables	Python ingestion extracts approved tables	Raw Parquet files
3	Raw Parquet files	PySpark standardization job cleans and normalizes data	Standardized Parquet files
4	Standardized Parquet files	PySpark curated build job creates facts and dimensions	Curated warehouse Parquet models
5	Curated and standardized Parquet files	DuckDB SQL runner executes validation scripts	SQL DQ results

Step	Input	Process	Output
6	Curated and standardized Parquet files	PySpark DQ runner executes profiling and validation	PySpark DQ results
7	Technical assets and DQ outputs	Governance documentation maps business and technical context	Glossary, catalog, CDEs, lineage, DQ rule catalog
8	Governance artifacts	OpenMetadata templates map assets to platform concepts	OpenMetadata documentation and config templates
9	Completed project artifacts	Agile/SDLC documents organize delivery evidence	User stories, test evidence, release notes

4.4 Logical Data Zones

The project uses three main data zones before validation and governance:

Zone	Description	Example Path Pattern
Raw zone	Source-like Parquet extracts from PostgreSQL	data/raw/<table_name>/load_date=YYYY-MM-DD/<table_name>.parquet
Standardized zone	Cleaned and schema-normalized Parquet outputs	data/standardized/<table_name>/load_date=YYYY-MM-DD/<table_name>.parquet
Curated zone	Warehouse-style fact and dimension outputs	data/curated/<model_name>/load_date=YYYY-MM-DD/<model_name>.parquet

The use of load-date partitioning gives each data movement step a repeatable and auditable structure. It also supports validation because row counts can be compared between source, raw, standardized, and curated layers for a specific load date.

4.5 Data Quality Architecture

The project includes two separate data quality layers: one SQL-based and one PySpark-based.

Data Quality Layer	Engine	Purpose
SQL Data Quality Framework	DuckDB	Practice SQL validation over Parquet without loading data into another database
PySpark Data Quality Framework	PySpark	Practice scalable profiling and validation using Spark DataFrame operations

The SQL DQ layer focuses on SQL scripts, reconciliation queries, control totals, and exception-record SQL output.

The PySpark DQ layer focuses on dataset profiling, column profiling, schema validation, Spark join-based referential integrity, Spark aggregation-based control totals, and bounded exception-record generation.

These two frameworks intentionally validate similar governance concerns through different implementation styles. This separation supports both SQL data quality engineering skills and PySpark data quality engineering skills.

4.6 Governance Architecture

The governance architecture sits on top of the technical pipeline. It does not replace the pipeline. It explains, classifies, catalogs, and governs the assets created by the pipeline.

Governance Area	Artifact
Governance operating model	governance/governance_policy.md
Business glossary	governance/glossary/business_terms.yml
Critical Data Elements	governance/metadata/critical_data_elements.yml
Data catalog	governance/catalog/data_assets.yml
Source-to-target mapping	governance/metadata/source_to_target_mapping.yml
Lineage mapping	governance/lineage/lineage_mapping.yml
Data quality rule catalog	governance/data_quality/dq_rules.yml
Stewardship issue examples	governance/stewardship/data_quality_issues.yml

This layer connects business terms, technical fields, curated datasets, data quality rules, ownership, classifications, and stewardship workflows.

4.7 OpenMetadata Architecture Position

OpenMetadata is positioned as a future governance-platform representation layer. The project does not claim that OpenMetadata was installed or that metadata ingestion was executed.

Instead, the project includes:

OpenMetadata Artifact	Purpose
openmetadata/README.md	Explains OpenMetadata role and governance mapping
openmetadata/ingestion_configs/postgres_metadata_ingestion.yml	Template for PostgreSQL metadata ingestion
openmetadata/ingestion_configs/curated_layer_metadata_ingestion.yml	Template for curated asset metadata representation

OpenMetadata Artifact	Purpose
openmetadata/ingestion_configs/lineage_ingestion.yml	Template for lineage ingestion concepts
openmetadata/local_runbook.md	Runbook for future local deployment and validation

OpenMetadata is used as an open-source counterpart for practicing Collibra-style governance concepts, including cataloging, glossary mapping, ownership, classifications, lineage, metadata ingestion design, and platform operating concepts.

4.8 Agile / SDLC Architecture Position

The Agile / SDLC layer documents how the completed technical and governance work would be planned, validated, and released in an enterprise environment.

Agile / SDLC Artifact	Purpose
agile/user_stories.md	Captures governance, catalog, lineage, DQ, and OpenMetadata requirements
agile/sprint_backlog.yml	Organizes work into a four-sprint delivery plan
agile/definition_of_done.md	Defines completion and approval criteria
agile/test_evidence.md	Captures validation evidence
agile/release_notes.md	Summarizes delivered capabilities

This layer is important because enterprise governance work requires more than technical implementation. It requires requirement tracking, acceptance criteria, test evidence, release discipline, and issue support.

4.9 Architecture Strengths

The architecture has several strengths:

Strength	Explanation
Layered design	Each layer has a clear purpose and output
Local reproducibility	The project can be demonstrated without external enterprise platforms
Synthetic data safety	No real customer data is used
Governance integration	Technical assets are connected to glossary, catalog, CDEs, lineage, and stewardship
Dual DQ approach	SQL and PySpark validation are both represented

Strength	Explanation
Audit-style evidence	Summary CSVs, DQ outputs, test evidence, and release notes support walkthroughs
Honest tooling boundaries	Collibra is not claimed, and OpenMetadata runtime ingestion is not overstated
Interview readiness	Each architecture layer maps directly to common Data Governance and Data Quality role expectations

---

5. Technology Stack

5.1 Technology Stack Overview

The project uses a practical local data engineering and governance stack. The tools were selected to support hands-on implementation while keeping the project reproducible on a local machine.

Category	Technology	Role in Project
Programming	Python	Data generation, ingestion, validation runners, automation support
Synthetic data	Faker	Generated realistic synthetic banking data
Source database	PostgreSQL	Simulated normalized banking source system
Container runtime	Docker Compose	Ran local PostgreSQL service
Data processing	PySpark	Standardization, schema handling, curated modeling, and PySpark DQ validation
File format	Parquet	Raw, standardized, curated, and DQ output storage
SQL validation engine	DuckDB	Queried Parquet files directly using SQL
Data quality	SQL and PySpark	Implemented validation checks, profiling, reconciliation, and exception outputs
Governance artifacts	YAML and Markdown	Documented glossary, catalog, CDEs, mappings, lineage, DQ rules, and policy
Governance platform concepts	OpenMetadata	Used as open-source counterpart for Collibra-style governance concepts
Version control	Git	Commit history, phase tags, and SDLC traceability

Category	Technology	Role in Project
Agile / SDLC documentation	Markdown and YAML	User stories, sprint backlog, Definition of Done, evidence, release notes

5.2 Python

Python is used across multiple parts of the project:

Python Usage	Description
Data generation	Faker generator creates synthetic banking data
PostgreSQL ingestion	Python extracts approved source/control tables
Database connectivity	SQLAlchemy and environment variables support PostgreSQL connection handling
Ingestion validation	Python validates table names, load dates, and Parquet read-back row counts
SQL DQ runner	Python initializes DuckDB views and executes SQL validation scripts
Operational logging	Python scripts generate logs and summary CSV files

Python is important because many data governance and data quality roles require scripting for extraction, validation, metadata preparation, and automation.

5.3 PostgreSQL

PostgreSQL acts as the simulated operational banking source system.

PostgreSQL Role	Description
Source database	Stores normalized synthetic banking data
Schema design	Supports primary keys, foreign keys, constraints, and indexes
Banking model	Represents customers, accounts, transactions, branches, products, risk scores, and governance control tables
Source validation	Provides the source row counts used for ingestion validation

PostgreSQL is used as a local OLTP-style source system, not as the final warehouse.

5.4 Faker

Faker is used to generate synthetic banking data. It allows the project to simulate realistic source records without using real customer, account, or transaction data.

Faker Usage	Description
Customer data	Names, emails, dates, and related customer attributes
Account data	Account-level banking records
Transaction data	Transaction activity records
Risk data	Customer risk score records
Governance support records	Sample DQ rules, stewardship issues, and source-to-target mappings

The use of Faker keeps the project safe, repeatable, and suitable for portfolio discussion.

5.5 Docker Compose

Docker Compose is used to run the local PostgreSQL service. This gives the project a repeatable database environment without requiring a manually installed database instance.

Docker Compose Usage	Description
PostgreSQL container	Runs the local source database
Local development support	Provides consistent database startup and configuration
Validation support	Allows schema loading and source data verification

Docker Compose is used only for local infrastructure support. It is not a production deployment model in this project.

5.6 Parquet

Parquet is the file format used for the data lake-style layers of the project.

Parquet Layer	Description
Raw Parquet	Source extracts landed from PostgreSQL
Standardized Parquet	Cleaned and schema-normalized outputs
Curated Parquet	Fact and dimension-style warehouse models
DQ result Parquet	PySpark DQ output files

Parquet was selected because it is a common columnar storage format used in data lake and lakehouse architectures. It also works well with PySpark and DuckDB.

5.7 PySpark

PySpark is used for both transformation and validation.

PySpark Usage	Description
Raw-to-standardized processing	Reads raw Parquet and writes standardized Parquet
Schema handling	Applies table-specific schema configuration and type casting
Data cleanup	Trims strings, normalizes codes/status fields, and handles blank values
Curated modeling	Builds curated fact and dimension datasets
Data quality validation	Profiles datasets, validates schemas, checks rules, joins facts to dimensions, reconciles counts, and validates control totals

PySpark is one of the most important technologies in the project because it demonstrates scalable data processing and validation patterns.

5.8 DuckDB

DuckDB is used as the SQL execution engine for data quality validation over local Parquet files.

DuckDB Usage	Description
SQL-over-Parquet	Queries standardized and curated Parquet files directly
Temporary views	Creates SQL views over Parquet datasets
SQL validation	Executes null, duplicate, accepted-value, referential-integrity, reconciliation, control-total, and exception-record queries
Local simplicity	Avoids loading curated Parquet data into another database

DuckDB was used only as a SQL validation engine. It was not introduced as a persistent warehouse layer.

5.9 SQL

SQL is used for data quality validation and reconciliation.

SQL Usage	Description
Null checks	Validate critical fields and keys
Duplicate checks	Validate logical primary keys and composite keys
Accepted-value checks	Validate controlled business fields
Referential-integrity checks	Validate fact-to-dimension relationships

SQL Usage	Description
Source-to-target reconciliation	Compare standardized and curated datasets
Control totals	Validate financial and count totals
Exception records	Return failed records for investigation

SQL remains central to the project because governance and data quality roles frequently require strong SQL validation, reconciliation, and audit evidence skills.

5.10 YAML and Markdown

YAML and Markdown are used to represent governance, metadata, and Agile/SDLC artifacts in a lightweight but structured way.

Format	Usage
YAML	Business glossary, CDEs, data assets, source-to-target mappings, lineage mappings, DQ rules, stewardship issues, OpenMetadata templates, sprint backlog
Markdown	Governance policy, OpenMetadata README, local runbook, user stories, Definition of Done, test evidence, release notes, project documentation

This approach allows the project to simulate governance platform artifacts without requiring enterprise tooling.

5.11 OpenMetadata

OpenMetadata is used as an open-source counterpart for practicing Collibra-style governance concepts.

OpenMetadata Concept	Project Representation
Glossary	Business glossary mapping
Data catalog	Data asset catalog mapping
Classifications and tags	CDE and classification mapping
Ownership	Owner and steward mapping
Lineage	Source-to-target and lineage mapping
Metadata ingestion	YAML configuration templates
Local operations	Deployment and validation runbook

OpenMetadata was not installed or executed in the completed project. The completed work includes integration design, safe ingestion templates, and a future local deployment runbook.

5.12 Git

Git is used to support SDLC-style delivery discipline.

Git Usage	Description
Commit tracking	Each major project component was committed
Phase tags	Tags were used to mark phase completion
Scope control	Git status and changed files were reviewed to prevent unintended changes
Delivery evidence	Commit history supports project traceability

Git helps show that the project was delivered through structured increments rather than as a single untracked code dump.

5.13 Agile / SDLC Documentation Tools

Agile and SDLC artifacts were created using local Markdown and YAML files.

Artifact	Format	Purpose
User stories	Markdown	Capture governance and DQ requirements
Sprint backlog	YAML	Organize work into a 4-sprint plan
Definition of Done	Markdown	Define completion criteria
Test evidence	Markdown	Capture validation evidence
Release notes	Markdown	Summarize delivered capabilities

These files simulate how governance work would be planned and delivered in JIRA or a similar enterprise Agile tool.

5.14 Technology Stack Summary

The selected technology stack supports the full project lifecycle:

Lifecycle Area	Technology Support
Data creation	Faker and Python
Source persistence	PostgreSQL and Docker Compose
Raw ingestion	Python, SQLAlchemy, Parquet
Processing	PySpark
Curated modeling	PySpark and Parquet
SQL validation	DuckDB and SQL

Lifecycle Area	Technology Support
Distributed-style validation	PySpark
Governance metadata	YAML and Markdown
Governance platform concepts	OpenMetadata documentation and templates
SDLC evidence	Git, Markdown, YAML

Together, these tools provide a complete local environment for practicing data engineering, data quality, data governance, cataloging, lineage, metadata management, and Agile delivery concepts.

6. Source Data Model

6.1 Source Data Model Overview

The source data model is the foundation of the Banking Data Governance & Data Quality Pipeline. It simulates a normalized banking operational source system in PostgreSQL and provides the upstream data required for ingestion, processing, curation, validation, cataloging, lineage, and governance practice.

The original starting point was a simple three-table structure containing customers, accounts, and transactions. That model was not sufficient for a realistic governance and data quality project because it lacked reference data, product context, branch relationships, risk records, governance control tables, data quality rule metadata, stewardship tracking, and source-to-target mapping support.

The final PostgreSQL source model was expanded into a 10-table normalized banking model. This model includes operational banking entities as well as governance support/control tables.

6.2 Source and Control Tables

Table	Table Type	Purpose
branches	Reference / master data	Stores banking branch information used by customers, accounts, and transactions
products	Reference / master data	Stores banking product information used by accounts
reference_codes	Reference data	Stores controlled values for status, type, category, and code-based fields
customers	Core banking entity	Stores customer master records
accounts	Core banking entity	Stores customer account records
transactions	Core banking activity	Stores banking transaction records
customer_risk_scores	Risk data	Stores risk score and risk band records for customers

Table	Table Type	Purpose
data_quality_rules	Governance control table	Stores sample data quality rule definitions
stewardship_issues	Governance control table	Stores sample stewardship issue records
source_to_target_mappings	Governance metadata table	Stores source-to-target mapping metadata

The inclusion of governance control tables was intentional. The project was designed not only to process banking data, but also to practice governance concepts such as data quality rule metadata, stewardship issue tracking, and source-to-target traceability.

6.3 Source Model Design Principles

The source model was designed around several key principles.

Design Principle	Implementation
Normalized structure	Banking entities are separated into customers, accounts, transactions, branches, products, and risk score tables
Referential integrity	Foreign keys connect customers, accounts, transactions, branches, products, risk scores, and stewardship issues
Stable identifiers	Business-facing identifiers such as customer number, account number, transaction reference, branch code, and product code are included
Governance readiness	Data quality rule metadata, stewardship issue records, and source-to-target mappings are included
Data quality readiness	Required fields, controlled values, and relationship structures support downstream validation
Lineage readiness	Source-to-target mapping records support future lineage and catalog documentation
Repeatable loading	The schema and generator support rerunnable local development and validation

6.4 Key Relationships

The source model includes the following core relationships:

Relationship	Description
customers.branch_id → branches.branch_id	A customer can be associated with a branch

Relationship	Description
accounts.customer_id → customers.customer_id	Each account belongs to a customer
accounts.product_id → products.product_id	Each account is associated with a banking product
accounts.branch_id → branches.branch_id	Each account can be associated with a branch
transactions.account_id → accounts.account_id	Each transaction is associated with an account
transactions.related_account_id → accounts.account_id	Transfer-style transactions can reference a related account
transactions.branch_id → branches.branch_id	Transactions can be associated with a branch when available
customer_risk_scores.customer_id → customers.customer_id	Risk score records belong to customers
stewardship_issues.dq_rule_id → data_quality_rules.dq_rule_id	Stewardship issues can reference data quality rules

These relationships allow the project to practice referential integrity checks, fact-to-dimension validation, source-to-target mapping, and lineage documentation.

6.5 Surrogate Keys and Business Identifiers

The source model uses both surrogate primary keys and stable business identifiers. This is important because technical processing and governance documentation often require different types of identifiers.

Entity	Surrogate Key	Business Identifier
Branch	branch_id	branch_code
Product	product_id	product_code
Customer	customer_id	customer_number
Account	account_id	account_number
Transaction	transaction_id	transaction_reference
Data Quality Rule	dq_rule_id	rule_code
Stewardship Issue	issue_id	issue_reference

Surrogate keys support database relationships and joins. Business identifiers support governance, cataloging, data lineage, issue tracking, and business-facing communication.

6.6 Synthetic Source Data Volumes

The Faker-based data generator populated all 10 PostgreSQL tables with valid synthetic banking data.

Table	Final Row Count
reference_codes	72
branches	10
products	6
customers	500
accounts	990
transactions	5,000
customer_risk_scores	742
data_quality_rules	12
stewardship_issues	8
source_to_target_mappings	25

The generated volumes are large enough to support meaningful ingestion, transformation, validation, reconciliation, and governance exercises while remaining small enough for local development.

6.7 Data Generation Logic

The data generator uses a single-run normalized loading approach. It loads parent tables before child tables to preserve foreign key relationships.

Generator Capability	Description
PostgreSQL connectivity	Connects to the local PostgreSQL source database
Safe reload behavior	Clears existing rows before loading new data
Parent-to-child loading	Loads reference and parent records before dependent records
Foreign key preservation	Uses generated IDs to maintain valid relationships
Transaction handling	Commits successful loads and rolls back on failure
Row-count output	Prints row counts after table loading
Resource cleanup	Closes database resources after completion

Generator Capability	Description
Environment configuration	Uses environment variables for connection settings

The generator requires PostgreSQL connection settings such as host, port, database, user, and password through environment variables. Local secrets are not committed to Git.

6.8 Governance Relevance of the Source Model

The source model was designed to support more than basic data movement. It provides the foundation for the governance and data quality layers created later in the project.

Governance Need	Source Model Support
Business glossary	Source fields map to business terms such as Customer, Account, Transaction, Transaction Amount, and Risk Score
Critical Data Elements	Key fields such as customer ID, account ID, transaction amount, transaction date, and risk score are available
Source-to-target mapping	The source_to_target_mappings table supports mapping practice
Data lineage	Source tables provide the starting point for lineage from PostgreSQL to curated models
Data quality rules	The data_quality_rules table supports data quality rule metadata practice
Stewardship workflow	The stewardship_issues table supports issue tracking examples
Cataloging	Source tables become cataloged assets in the governance layer

The result is a source model that supports technical processing and governance documentation at the same time.

---

7. Data Ingestion Architecture

7.1 Ingestion Architecture Overview

The data ingestion layer moves data from the PostgreSQL banking source system into a local raw Parquet zone. This is the first data movement step in the pipeline.

The ingestion flow is:

PostgreSQL Banking Source System → Python Ingestion Scripts

→ **Raw** **Parquet** **Zone**

→ **Ingestion Logs and Summary CSV Evidence**

The ingestion layer was implemented using Python. It extracts all approved source and control tables from PostgreSQL, writes each table to Parquet format, partitions the output by load date, and validates that the written Parquet row counts match the extracted source row counts.

7.2 Ingestion Objectives

The ingestion layer was designed to meet the following objectives:

Objective	Implementation
Extract source data	Read all approved PostgreSQL source/control tables
Create raw data zone	Write table-level Parquet files under data/raw
Preserve load context	Use load_date=YYYY-MM-DD partition folders
Validate extraction	Compare extracted row count to Parquet read-back row count
Improve reliability	Validate table names and load-date format
Support traceability	Generate ingestion logs and summary CSV files
Protect configuration	Use environment variables for PostgreSQL connection settings
Keep repository clean	Exclude runtime data and logs from Git tracking

7.3 Python Ingestion Components

The ingestion layer includes two main Python components.

Component	Purpose
python/ingestion/db_connection.py	Handles PostgreSQL connection creation using environment variables
python/ingestion/extract_postgres_to_raw.py	Extracts approved PostgreSQL tables and writes raw Parquet outputs

The connection logic was separated from the extraction logic to make the code easier to maintain and reuse. The connection utility loads environment variables, validates required PostgreSQL settings, creates a SQLAlchemy engine, and avoids logging sensitive credentials.

The extraction script performs the actual ingestion work. It validates command-line inputs, checks table names against an approved list, validates load-date format, extracts data from PostgreSQL, writes Parquet files, reads the Parquet files back, compares row counts, writes logs, and creates a summary CSV.

7.4 Approved Tables Extracted

The ingestion process extracts the same 10 approved source/control tables created in the source model:

Table
branches
products
reference_codes
customers
accounts
transactions
customer_risk_scores
data_quality_rules
stewardship_issues
source_to_target_mappings

The extraction job validates requested tables against this approved list. Invalid table names are rejected before extraction. This prevents accidental extraction of unapproved database objects.

7.5 Raw Output Design

The raw zone uses local Parquet files partitioned by load date.

Design Element	Value
Raw root folder	data/raw
Partition key	load_date
Date format	YYYY-MM-DD
File format	Parquet
Output pattern	data/raw/<table_name>/load_date=YYYY-MM-DD/<table_name>.parquet

Example output paths:

Table	Raw Output Path
customers	data/raw/customers/load_date=2026-06-16/customers.parquet
accounts	data/raw/accounts/load_date=2026-06-16/accounts.parquet

Table	Raw Output Path
transactions	data/raw/transactions/load_date=2026-06-16/transactions.parquet

This design simulates a common raw data lake landing pattern where data is organized by source table and ingestion date.

7.6 Ingestion Validation Controls

The ingestion layer includes several validation controls.

Validation Control	Purpose
Table whitelist validation	Prevents extraction of unapproved tables
Load-date validation	Requires YYYY-MM-DD format
Parquet read-back verification	Confirms the written Parquet file is readable
Row-count comparison	Confirms extracted row count matches Parquet row count
Error handling	Exits with failure when extraction or validation fails
Summary CSV	Provides run-level evidence for each extracted table
Logging	Captures ingestion progress and issues

These controls make the ingestion process more reliable and closer to a production-style extraction process.

7.7 Ingestion Summary Output

The ingestion job produces a summary CSV with the following fields:

Field	Description
table_name	Name of the extracted table
status	Extraction status
extracted_row_count	Row count read from PostgreSQL
parquet_row_count	Row count read back from the Parquet file
output_path	Raw Parquet output path
error_message	Error details when a table fails

This summary file provides lightweight operational evidence for ingestion validation.

7.8 Final Ingestion Validation Results

The final full extraction successfully extracted all 10 tables. The extracted row counts matched the Parquet read-back row counts.

Table	Extracted Row Count	Parquet Row Count	Status
branches	10	10	SUCCESS
products	6	6	SUCCESS
reference_codes	72	72	SUCCESS
customers	500	500	SUCCESS
accounts	990	990	SUCCESS
transactions	5,000	5,000	SUCCESS
customer_risk_scores	742	742	SUCCESS
data_quality_rules	12	12	SUCCESS
stewardship_issues	8	8	SUCCESS
source_to_target_mappings	25	25	SUCCESS

The ingestion validation confirmed that all source records were successfully landed in the raw Parquet zone without row-count loss.

7.9 Ingestion Layer Outcome

The ingestion layer created a controlled raw landing foundation for the rest of the project. It provided:

Outcome	Result
PostgreSQL extraction	Completed for all 10 approved tables
Raw Parquet generation	Completed
Load-date partitioning	Completed
Logging	Completed
Summary CSV reporting	Completed
Invalid table rejection	Completed
Invalid load-date rejection	Completed
Parquet row-count verification	Completed

Outcome	Result
Runtime output handling	Runtime folders excluded from Git

This layer prepared the project for PySpark standardization by producing reliable raw Parquet inputs.

---

8. PySpark Processing Layer

8.1 PySpark Processing Overview

The PySpark processing layer reads raw Parquet files from the ingestion layer, applies standardization and schema handling, and writes clean standardized Parquet outputs. This is the first Spark-based transformation step in the project.

The processing flow is:

PostgreSQL Banking Source System
→ **Python Ingestion Layer**
→ **Raw Parquet Zone**
→ **PySpark Processing Layer**
→ **Standardized Parquet Zone**

The purpose of this layer is to move the pipeline from raw source extracts into a cleaner, schema-aware, and traceable standardized layer.

8.2 Processing Objectives

The PySpark processing layer was designed to meet the following objectives:

Objective	Implementation
Read raw data	Reads raw Parquet files by table and load date
Validate inputs	Validates approved table names and load-date format
Standardize columns	Converts column names to lowercase snake_case
Clean string values	Trims strings and converts blank strings to null
Normalize selected values	Lowercases email fields and uppercases code/status/type/category fields
Enforce schemas	Applies table-specific schema configuration
Validate required columns	Checks required columns after standardization
Cast data types	Applies table-specific type casting
Add traceability	Adds processing metadata columns

Objective	Implementation
Write standardized outputs	Writes standardized Parquet files by table and load date
Validate row counts	Compares raw and standardized row counts
Generate evidence	Writes summary CSV logs

8.3 PySpark Processing Components

The PySpark processing layer includes the following files:

Component	Purpose
spark/utils/spark_session.py	Creates a reusable local Spark session
spark/utils/schema_config.py	Defines table-specific schema and type handling configuration
spark/jobs/process_raw_to_standardized.py	Runs the raw-to-standardized processing job

These components separate Spark session creation, schema configuration, and processing logic. This makes the processing layer easier to maintain and extend.

8.4 Tables Processed

The PySpark processing layer processes the same 10 approved source/control tables extracted by the Python ingestion layer.

Table
branches
products
reference_codes
customers
accounts
transactions
customer_risk_scores
data_quality_rules
stewardship_issues
source_to_target_mappings

The processing job validates the requested tables against this approved list before reading or writing data.

8.5 Standardized Output Design

The standardized zone uses local Parquet files partitioned by load date.

Design Element	Value
Standardized root folder	data/standardized
Partition key	load_date
Date format	YYYY-MM-DD
File format	Parquet
Output pattern	data/standardized/<table_name>/load_date=YYYY-MM-DD/<table_name>.parquet

Example output paths:

Table	Standardized Output Path
customers	data/standardized/customers/load_date=2026-06-16/customers.parquet
accounts	data/standardized/accounts/load_date=2026-06-16/accounts.parquet
transactions	data/standardized/transactions/load_date=2026-06-16/transactions.parquet

This design separates raw source extracts from standardized, schema-normalized outputs.

8.6 Standardization Rules

The processing layer applies a consistent set of standardization rules.

Rule	Purpose
Lowercase snake_case columns	Creates consistent column naming
String trimming	Removes leading and trailing spaces
Blank-string-to-null conversion	Prevents blank values from being treated as valid text
Email lowercasing	Standardizes email fields for matching and governance
Code/status/type/category uppercasing	Standardizes controlled categorical fields
Source-system field normalization	Keeps source-related codes consistent
Preferred column ordering	Produces consistent output structure

These transformations support downstream modeling, data quality validation, and cataloging.

8.7 Schema Handling and Type Normalization

The standardized layer includes table-specific schema handling. This gives each approved table an expected processing contract.

Schema Handling Feature	Description
Table-specific schema configuration	Defines expected structure for each approved table
Required-column validation	Fails clearly when required columns are missing
Type casting	Casts IDs, amounts, balances, scores, dates, timestamps, booleans, and strings
Preferred output ordering	Produces predictable standardized datasets
Better failure handling	Captures missing required columns and processing errors in the summary output

This makes the standardized layer more reliable than a simple raw copy process.

8.8 Metadata Columns

The standardized layer adds metadata columns for processing traceability.

Metadata Column	Purpose
source_table	Identifies the source table processed by the Spark job
source_load_date	Preserves the source load date
standardized_load_date	Identifies the standardized load date
standardized_at_utc	Captures the UTC processing timestamp

Some source data already contained a business column named source_table. To avoid ambiguity, those business values were preserved as source_table_name, while the processing metadata column remained source_table.

8.9 Summary CSV Reporting

The PySpark processing job generates a timestamped summary CSV under logs/spark.

The summary includes:

Field	Description
table_name	Name of the processed table

Field	Description
status	Processing status
raw_row_count	Number of rows read from raw Parquet
standardized_row_count	Number of rows written to standardized Parquet
raw_column_count	Number of columns in the raw dataset
standardized_column_count	Number of columns in the standardized dataset
missing_required_columns	Missing required columns, if any
input_path	Raw input path
output_path	Standardized output path
error_message	Error details when processing fails

This summary provides operational evidence for raw-to-standardized processing.

8.10 Final PySpark Processing Validation Results

The full PySpark standardized processing run succeeded for all 10 approved tables.

Table	Raw Row Count	Standardized Row Count	Status
branches	10	10	SUCCESS
products	6	6	SUCCESS
reference_codes	72	72	SUCCESS
customers	500	500	SUCCESS
accounts	990	990	SUCCESS
transactions	5,000	5,000	SUCCESS
customer_risk_scores	742	742	SUCCESS
data_quality_rules	12	12	SUCCESS
stewardship_issues	8	8	SUCCESS
source_to_target_mappings	25	25	SUCCESS

The validation confirmed that all raw row counts matched standardized row counts. Required columns were present, standardized Parquet outputs were written, and summary evidence was generated.

A local Windows Spark warning related to winutils.exe appeared during execution, but the Spark job completed successfully and the warning did not affect the output.

8.11 PySpark Processing Layer Outcome

The PySpark processing layer successfully created the standardized data foundation for the curated warehouse layer.

Outcome	Result
Raw Parquet reading	Completed
Standardized Parquet writing	Completed
Load-date partitioning	Completed
Table whitelist validation	Completed
Load-date validation	Completed
Column standardization	Completed
String cleanup	Completed
Metadata column creation	Completed
Required-column checks	Completed
Type casting	Completed
Preferred output column ordering	Completed
Row-count validation	Completed
Summary CSV reporting	Completed

The standardized layer improved consistency, traceability, and schema control before curated warehouse modeling.

---

9. Curated Warehouse Layer

9.1 Curated Warehouse Overview

The curated warehouse layer transforms standardized banking data into analytics-ready fact and dimension models. This layer represents the movement from standardized operational data into governed, reporting-friendly datasets.

The curated processing flow is:

PostgreSQL Banking Source System
→ **Python Ingestion Layer**
→ **Raw Parquet Zone**
→ **PySpark Standardized Processing Layer**
→ **Standardized Parquet Zone**
→ **Curated Warehouse Layer**

The curated warehouse layer is designed for reporting, analytics, governance practice, data quality validation, source-to-target mapping, and lineage documentation.

9.2 Curated Modeling Objectives

The curated warehouse layer was designed to meet the following objectives:

Objective	Implementation
Create analytics-ready models	Build fact and dimension-style curated datasets
Define business purpose	Document the purpose of each curated model
Define grain	Specify what one row represents in each model
Define logical keys	Document logical primary keys and foreign keys
Support governance	Connect curated models to business terms, CDEs, catalog, and lineage
Support DQ validation	Provide targets for SQL and PySpark DQ frameworks
Preserve row movement	Track row counts and join-loss metrics
Generate evidence	Write model-level summary CSV output

9.3 Curated Warehouse Components

The curated warehouse layer includes design configuration, documentation, and a PySpark build job.

Component	Purpose
spark/utils/model_config.py	Central configuration for curated model definitions
docs/data_model_design.md	Documentation of model purpose, grain, keys, relationships, and source tables
spark/jobs/build_curated_models.py	PySpark job that builds curated models from standardized Parquet inputs

The model configuration defines exactly seven curated models. Each model includes model name, model type, business purpose, grain, source tables, logical primary key, logical foreign keys, target columns, and notes where needed.

9.4 Curated Models Built

The curated warehouse layer contains seven models.

Model	Type	Grain	Logical Primary Key	Source Table(s)	Final Row Count
dim_customer	Dimension	One row per customer	customer_id	customers	500
dim_account	Dimension	One row per account	account_id	accounts	990
dim_branch	Dimension	One row per branch	branch_id	branches	10
dim_product	Dimension	One row per product	product_id	products	6
fact_transaction	Fact	One row per transaction	transaction_id	transactions, accounts	5,000
fact_customer_risk_score	Fact	One row per customer risk score record	Risk score record key / customer risk score grain	customer_risk_scores	742
fact_daily_account_balance	Fact	One row per account per balance date	account_id, balance_date	accounts, transactions	990

These models create a simplified banking warehouse structure suitable for analytics, governance, and data quality validation.

9.5 Dimension Models

9.5.1 dim_customer

Attribute	Description
Purpose	Supports customer-level reporting, customer governance, customer segmentation, and downstream DQ validation
Grain	One row per customer
Logical primary key	customer_id

Attribute	Description
Logical foreign key	branch_id → dim_branch.branch_id
Source table	customers
Final row count	500

dim_customer is a core governed dimension because customer identity, customer number, email, and date of birth are important governance fields. Several of these fields are also represented as Critical Data Elements in the governance layer.

9.5.2 dim_account

Attribute	Description
Purpose	Supports account-level analytics, ownership analysis, product relationship analysis, and transaction relationship mapping
Grain	One row per account
Logical primary key	account_id
Logical foreign keys	customer_id → dim_customer.customer_id, product_id → dim_product.product_id, branch_id → dim_branch.branch_id
Source table	accounts
Final row count	990

dim_account connects customers, products, and branches. It is a key model for account ownership analysis and transaction context.

9.5.3 dim_branch

Attribute	Description
Purpose	Supports branch-level reporting, customer branch assignment, account branch assignment, and transaction branch analysis
Grain	One row per branch
Logical primary key	branch_id
Source table	branches

Attribute	Description
Final row count	10

dim_branch provides organizational and location context for customers, accounts, and transactions.

9.5.4 dim_product

Attribute	Description
Purpose	Supports banking product analysis, product ownership reporting, and account-product relationship mapping
Grain	One row per product
Logical primary key	product_id
Source table	products
Final row count	6

dim_product provides product context for account and transaction reporting.

9.6 Fact Models

9.6.1 fact_transaction

Attribute	Description
Purpose	Supports transaction-level analysis, financial activity reporting, reconciliation, fraud review, and DQ validation
Grain	One row per transaction
Logical primary key	transaction_id
Logical foreign keys	account_id, customer_id, branch_id, product_id
Source tables	transactions, accounts
Final row count	5,000
Join-loss result	0 records lost to join

fact_transaction joins transaction records to account records to bring in account-related dimensional keys such as customer_id and product_id. This model is central to the project because it supports financial

control totals, transaction amount validation, referential integrity checks, source-to-target reconciliation, and lineage examples.

9.6.2 fact_customer_risk_score

Attribute	Description
Purpose	Supports customer risk monitoring, risk trend analysis, governance over risk-score fields, and DQ validation
Grain	One row per customer risk score record
Logical foreign key	customer_id → dim_customer.customer_id
Source table	customer_risk_scores
Final row count	742

fact_customer_risk_score supports risk-related governance and validation. Fields such as risk score and risk band are included in the governance layer as Critical Data Elements.

9.6.3 fact_daily_account_balance

Attribute	Description
Purpose	Supports account-day level balance summary analysis
Grain	One row per account per balance date
Logical composite key	account_id, balance_date
Logical foreign keys	account_id, customer_id, product_id, branch_id
Source tables	accounts, transactions
Final row count	990

fact_daily_account_balance is a derived account-day summary based on the available standardized account and transaction data. It is useful for practicing fact modeling and balance-related validation. It should not be interpreted as an official banking ledger balance table unless a true daily balance snapshot source is added in the future.

9.7 Logical Keys and Parquet Constraint Limitation

The curated layer uses logical primary keys and logical foreign keys. Because the curated models are stored as Parquet files, the storage format does not enforce database constraints in the same way a relational database would.

For this reason, keys are handled through:

Key Control	Implementation
Logical primary key documentation	Defined in the model configuration and data model documentation
Logical foreign key documentation	Defined in the model configuration and relationship documentation
Duplicate key validation	Checked during curated model build and later DQ validation
Referential integrity validation	Checked later through SQL and PySpark DQ frameworks
Source-to-target reconciliation	Checked after curated modeling

This is an important design point. The curated layer documents keys and validates them through checks rather than relying on Parquet to enforce constraints.

9.8 Curated Output Design

The curated outputs are written as local Parquet files partitioned by load date.

Design Element	Value
Curated root folder	data/curated
Partition key	load_date
Date format	YYYY-MM-DD
File format	Parquet
Output pattern	data/curated/<model_name>/load_date=YYYY-MM-DD/<model_name>.parquet

Curated output paths for the completed load include:

Model	Curated Output Path
dim_customer	data/curated/dim_customer/load_date=2026-06-16/dim_customer.parquet
dim_account	data/curated/dim_account/load_date=2026-06-16/dim_account.parquet
dim_branch	data/curated/dim_branch/load_date=2026-06-16/dim_branch.parquet
dim_product	data/curated/dim_product/load_date=2026-06-16/dim_product.parquet
fact_transaction	data/curated/fact_transaction/load_date=2026-06-16/fact_transaction.parquet
fact_customer_risk_score	data/curated/fact_customer_risk_score/load_date=2026-06-16/fact_customer_risk_score.parquet

Model	Curated Output Path
fact_daily_account_balance	data/curated/fact_daily_account_balance/load_date=2026-06-16/fact_daily_account_balance.parquet

These outputs are runtime data files and were not committed to Git.

9.9 Curated Build Validation

The curated model build job includes lightweight validation and sanity checks.

Validation Type	Purpose
Model whitelist validation	Ensures only approved curated models are built
Load-date validation	Requires valid YYYY-MM-DD load date
Row-count tracking	Captures output row count for each model
Duplicate logical key checks	Detects duplicate keys in curated outputs
Null foreign key counts	Captures potential relationship issues
Join-loss tracking	Confirms joins do not unexpectedly drop records
Summary CSV generation	Produces evidence for model build results

The summary CSV includes model name, status, source tables, output row count, duplicate key count, null foreign key count, input paths, output path, and error message.

9.10 Final Curated Build Results

The full curated model build succeeded for all seven models.

Model	Rows	Duplicate Keys	Null FK Rows	Status
dim_customer	500	0	0	SUCCESS
dim_account	990	0	0	SUCCESS
dim_branch	10	0	0	SUCCESS
dim_product	6	0	0	SUCCESS
fact_transaction	5,000	0	4,495	SUCCESS
fact_customer_risk_score	742	0	0	SUCCESS
fact_daily_account_balance	990	0	0	SUCCESS

The fact_transaction null foreign key count was informational. It included optional relationship fields such as branch and related account references, where null values can be valid depending on the transaction context. It was not treated as a curated build failure.

The fact_transaction enrichment produced 5,000 rows with 0 records lost to join. This confirmed that transaction-to-account enrichment did not drop records.

9.11 Curated Warehouse Outcome

The curated warehouse layer successfully created governed, analytics-ready datasets from standardized banking data.

Outcome	Result
Curated model design	Completed
Central model configuration	Completed
Data model documentation	Completed
Dimension models	Completed
Fact models	Completed
Standardized Parquet reading	Completed
Curated Parquet writing	Completed
Load-date partitioning	Completed
Model whitelist validation	Completed
Row-count sanity checks	Completed
Duplicate key checks	Completed
Null FK count tracking	Completed
Join-loss tracking	Completed
Summary CSV reporting	Completed

This layer is one of the most important parts of the project because it provides the primary target assets for SQL data quality validation, PySpark data quality validation, cataloging, lineage mapping, Critical Data Element mapping, and OpenMetadata representation.

10. SQL Data Quality Framework

10.1 SQL Data Quality Overview

The SQL Data Quality Framework validates the curated warehouse layer using SQL-based checks executed through DuckDB. This framework was created after the raw, standardized, and curated layers were already built. Its purpose is to provide audit-style validation over curated data assets and confirm that key business and technical rules are satisfied.

The SQL validation layer validates the curated warehouse outputs and compares them to standardized/source-like datasets where reconciliation is required. The validation flow is:

PostgreSQL Banking Source System
→ **Python Ingestion Layer**
→ **Raw Parquet Zone**
→ **PySpark Standardized Layer**
→ **Curated Warehouse Layer**
→ **DuckDB SQL Validation Layer**
→ **SQL Data Quality Results and Logs**

DuckDB was selected because the project stores standardized and curated datasets as Parquet files. DuckDB can query Parquet files directly using SQL, which avoids unnecessary database-loading complexity and keeps the SQL validation layer focused on data quality checks rather than infrastructure setup.

10.2 SQL Validation Design

The SQL framework uses an in-memory DuckDB connection and creates temporary SQL views over the standardized and curated Parquet outputs. DuckDB is used only as a SQL execution engine. It is not introduced as a persistent warehouse.

Design Area	Implementation
SQL engine	DuckDB
Data source	Standardized and curated Parquet files
Execution style	Python runner executes SQL scripts
View strategy	Temporary DuckDB views over Parquet datasets
Output format	CSV result files and summary logs
Failure behavior	Runner/configuration/execution errors fail the run
Validation target	Curated warehouse models
Reconciliation source	Standardized/source-like Parquet datasets

This design keeps the validation process simple, SQL-first, and local.

10.3 SQL Validation Components

The SQL validation framework includes a Python runner and multiple SQL validation scripts.

Component	Purpose
python/validation/run_sql_validation.py	Runs SQL validation scripts through DuckDB
sql/validation/01_null_checks.sql	Validates required fields and critical keys for nulls
sql/validation/02_duplicate_checks.sql	Validates duplicate logical keys
sql/validation/03_accepted_values_checks.sql	Validates controlled values
sql/validation/04_referential_integrity_checks.sql	Validates fact-to-dimension relationships
sql/validation/05_source_target_reconciliation.sql	Reconciles standardized/source-like data to curated outputs
sql/validation/06_control_totals.sql	Validates financial and count-based totals
sql/validation/07_exception_records.sql	Produces bounded exception records for review
docs/sql_data_quality_validation.md	Documents the SQL DQ framework and validation categories

The SQL runner creates views for the seven curated models and selected standardized datasets. It then executes the validation SQL files and writes results to the data quality results folder.

10.4 Curated Models Validated

The SQL framework validates the seven curated warehouse models created in the curated layer.

Curated Model	Validation Purpose
dim_customer	Customer key, required field, accepted value, duplicate, and relationship validation
dim_account	Account key, customer/product/branch relationship, duplicate, and accepted value validation
dim_branch	Branch key and duplicate validation
dim_product	Product key and duplicate validation
fact_transaction	Transaction key, account/customer/product/branch relationship, amount, reconciliation, and control-total validation
fact_customer_risk_score	Customer risk relationship, risk score, and risk band validation
fact_daily_account_balance	Account-day grain, account relationship, and balance-related validation

The standardized/source-like datasets are also used for source-to-target reconciliation and control-total comparison.

10.5 SQL Validation Categories

The SQL Data Quality Framework includes seven major validation categories.

Validation Category	Purpose	Example Checks
Null checks	Confirm required fields and critical keys are populated	Customer ID, account ID, transaction ID, transaction amount
Duplicate logical-key checks	Confirm logical keys are unique	Duplicate customer ID, account ID, transaction ID, composite account-day keys
Accepted-value checks	Confirm categorical fields contain expected values	Account status, transaction type, risk band
Referential-integrity checks	Confirm fact records map to valid dimensions	Transaction account ID to account dimension, customer ID to customer dimension
Source-to-target reconciliation	Confirm records were not lost during transformation	Standardized customers to dim_customer, standardized transactions to fact_transaction
Control-total validation	Confirm financial and count totals remain consistent	Transaction amount totals, grouped transaction counts
Exception-record queries	Return failed records for investigation	Orphan keys, duplicate transaction IDs, missing target records

These checks represent common controls used in governed data platforms, financial reporting, audit preparation, and data quality programs.

10.6 Standard SQL Result Structure

Most SQL validation scripts return a standard result structure.

Field	Description
check_category	Validation category such as null check, duplicate check, or reconciliation
check_name	Specific validation rule name
table_name	Dataset being validated
column_name	Column being validated, where applicable
severity	Business or technical severity level

Field	Description
failed_record_count	Number of failed records
status	Pass/fail result
details	Additional validation details

Exception-record queries use a different result shape because they return actual failed records rather than only summary counts. The SQL runner was updated to support both summary-style validation results and exception-record result schemas.

10.7 Source-to-Target Reconciliation

Source-to-target reconciliation confirms that the curated layer did not unexpectedly lose records during transformation from the standardized layer.

Reconciliation Check	Purpose
Standardized customers to dim_customer	Confirms customer row movement
Standardized accounts to dim_account	Confirms account row movement
Standardized branches to dim_branch	Confirms branch row movement
Standardized products to dim_product	Confirms product row movement
Standardized transactions to fact_transaction	Confirms transaction row movement
Standardized customer risk scores to fact_customer_risk_score	Confirms risk score row movement
Source transaction ID to curated transaction ID	Confirms transaction key preservation
Source account ID to curated account ID	Confirms account key preservation
Source customer ID to curated customer ID	Confirms customer key preservation

This validation is important because a curated model can have correct logic in some areas but still lose records during joins, filtering, or transformations.

10.8 Control Totals

Control-total validation confirms that financial totals and grouped counts remain consistent between standardized/source-like data and curated outputs. This is especially important in banking and financial reporting because row-count reconciliation alone is not enough.

Control-Total Area	Purpose
Total transaction amount	Confirms transaction amount values remain consistent

Control-Total Area	Purpose
Transaction counts by available transaction fields	Confirms grouped transaction distribution remains consistent
Account counts by available account fields	Confirms account grouping consistency
Customer counts by branch where available	Confirms branch-level customer grouping consistency
Balance-related comparisons where available	Confirms balance-related consistency

A dataset can have the correct row count but incorrect financial values. Control totals help detect that type of issue.

10.9 Exception Records

Exception-record queries return actual records that fail a validation rule. This supports investigation by data stewards, data quality engineers, and data engineers.

Exception Type	Purpose
Orphan transaction account IDs	Identify transactions that do not map to valid accounts
Orphan transaction customer IDs	Identify transactions that do not map to valid customers
Orphan transaction product IDs	Identify transactions that do not map to valid products
Duplicate transaction IDs	Identify duplicate transaction keys
Null critical keys	Identify missing key values
Invalid transaction amount records	Identify invalid financial values
Source transactions missing from fact_transaction	Identify transaction records lost during curation
Source accounts missing from dim_account	Identify account records lost during curation
Source customers missing from dim_customer	Identify customer records lost during curation

Each exception query is bounded to avoid producing large result sets. This simulates a realistic review process where failed records are sampled, investigated, assigned, remediated, validated, and closed.

10.10 SQL Validation Results

The SQL validation framework completed successfully.

Validation Area	Result
Null checks	8 checks, 0 failed
Duplicate checks	7 checks, 0 failed
Accepted-value checks	9 checks, 0 failed
Referential-integrity checks	14 checks, 0 failed
Source-to-target reconciliation	9 checks, 0 failed
Control totals	7 checks, 0 failed
Exception records	0 rows
Full SQL suite	54 checks, 0 failed

The final SQL validation result demonstrates that the curated warehouse layer passed the implemented SQL data quality rules, reconciliation checks, control-total validations, and exception-record checks.

10.11 SQL Data Quality Outcome

The SQL Data Quality Framework provides measurable governance evidence for the curated warehouse layer.

Outcome	Result
DuckDB SQL validation foundation	Completed
SQL runner	Completed
Curated Parquet views	Completed
Standardized Parquet views	Completed
Null checks	Completed
Duplicate checks	Completed
Accepted-value checks	Completed
Referential-integrity checks	Completed
Source-to-target reconciliation	Completed
Control totals	Completed
Exception-record queries	Completed

Outcome	Result
SQL result output	Completed
Summary logging	Completed
Final validation result	54 checks, 0 failed

This framework demonstrates the practical SQL validation work behind enterprise data quality programs, regulatory reporting support, reconciliation controls, audit readiness, and trusted curated datasets.

---

11. PySpark Data Quality Framework

11.1 PySpark Data Quality Overview

The PySpark Data Quality Framework provides a separate validation layer using Spark-based processing. It complements the SQL Data Quality Framework but does not replace it.

The purpose of this layer is to practice scalable data profiling and validation using PySpark DataFrame operations, filters, joins, grouping, aggregations, and exception-record generation.

The PySpark validation flow is:

PostgreSQL Banking Source System
→ **Python Ingestion Layer**
→ **Raw Parquet Zone**
→ **PySpark Standardized Layer**
→ **Curated Warehouse Layer**
→ **SQL Data Quality Validation Layer**
→ **PySpark Data Quality Validation Layer**

The framework validates the same governed curated assets but uses Spark logic instead of SQL scripts.

11.2 PySpark DQ Objectives

The PySpark DQ framework was designed to meet the following objectives:

Objective	Implementation
Profile curated datasets	Capture row counts, column counts, missing columns, and required-column status
Profile columns	Capture null count, non-null count, and distinct count
Validate schemas	Compare actual columns to expected configuration
Validate required columns	Confirm required columns exist

Objective	Implementation
Validate duplicate keys	Group by logical keys and detect duplicates
Validate accepted values	Check controlled categorical fields
Validate numeric ranges	Check financial and risk-related numeric fields
Validate relationships	Use Spark joins to detect orphan records
Reconcile source to target	Compare standardized and curated row counts
Validate control totals	Compare financial totals and grouped counts
Generate exception records	Create bounded failed-record outputs
Produce evidence	Write CSV and Parquet DQ outputs and summary logs

11.3 PySpark DQ Components

The PySpark DQ framework includes configuration, execution logic, documentation, and output generation.

Component	Purpose
spark/dq/dq_config.py	Defines DQ configuration for curated datasets
spark/dq/run_pyspark_dq_validation.py	Runs PySpark profiling, validation, reconciliation, and exception-record generation
docs/pyspark_data_quality_validation.md	Documents the PySpark DQ framework
data/dq_results/pyspark/load_date=YYYY-MM-DD/	Runtime output location for DQ results
logs/dq/	Runtime output location for DQ summary logs

The configuration-driven design makes the framework easier to extend. Datasets, expected columns, required columns, logical keys, profile columns, and rule categories are defined centrally.

11.4 Target Datasets

The framework validates the seven curated warehouse datasets.

Curated Dataset	Validation Purpose
dim_customer	Customer profile, schema, required fields, duplicate keys, and accepted values

Curated Dataset	Validation Purpose
dim_account	Account profile, schema, relationship fields, account status, and duplicate keys
dim_branch	Branch profile, schema, and duplicate keys
dim_product	Product profile, schema, and duplicate keys
fact_transaction	Transaction profile, financial rules, referential integrity, reconciliation, and control totals
fact_customer_risk_score	Risk score profile, range checks, risk band checks, and customer relationship validation
fact_daily_account_balance	Account-day balance profile, composite grain checks, relationship validation, and balance checks

The framework also uses standardized/source-like datasets where needed for reconciliation and control-total comparisons.

11.5 Dataset-Level Profiling

Dataset-level profiling captures high-level information about each curated dataset.

Profile Metric	Purpose
Row count	Measures dataset volume
Column count	Measures actual dataset structure
Expected column count	Compares actual structure to expected configuration
Missing expected columns	Identifies expected columns not present in the dataset
Extra columns	Identifies columns not defined in the expected configuration
Missing required columns	Identifies required governance or processing fields that are absent

Dataset profiling provides quick visibility into whether a curated asset is structurally complete and ready for validation.

11.6 Column-Level Profiling

Column-level profiling captures data quality indicators for configured columns.

Column Metric	Purpose
Null count	Identifies missing values

Column Metric	Purpose
Non-null count	Confirms populated values
Distinct count	Supports uniqueness and distribution analysis

Column profiling is useful for governance because it gives data stewards and engineers evidence about completeness, uniqueness, and field-level quality patterns.

11.7 Rule Validation Categories

The PySpark DQ framework includes multiple validation categories.

Rule Category	Purpose
Schema validation	Confirms expected columns exist
Required-column validation	Confirms required fields are present
Duplicate logical-key validation	Detects duplicate logical keys
Accepted-value validation	Validates controlled categorical values
Range validation	Validates numeric and financial values
Referential-integrity validation	Uses Spark joins to detect orphan relationships
Source-to-target reconciliation	Compares standardized and curated row counts
Control-total validation	Compares financial totals and grouped counts
Exception-record generation	Captures failed records for review

This rule set creates a Spark-based quality framework aligned with governance and audit-readiness needs.

11.8 Accepted-Value Checks

Accepted-value checks validate controlled fields where expected values are known.

Field Type	Example Validation
Customer status	Confirms values fall within approved customer statuses
Account status	Confirms values fall within approved account statuses
Account type	Confirms values fall within expected account types where available
Transaction type	Confirms values fall within approved transaction types
Transaction status	Confirms values fall within approved transaction statuses where available

Field Type	Example Validation
Risk band or risk rating	Confirms risk classification values are valid

These checks help prevent reporting and governance issues caused by inconsistent categorical values.

11.9 Range Checks

Range checks validate numeric and financial fields.

Field Area	Example Validation
Transaction amount	Amount should be greater than zero
Account balance	Balance should remain within configured reasonable ranges
Customer risk score	Risk score should remain within the configured valid scale
Daily account balance	Balance-related fields should be present and within expected ranges where available

Range validation is important for banking data because invalid numeric values can affect financial reporting, risk analysis, fraud detection, and downstream analytics.

11.10 Referential-Integrity Checks

Referential-integrity checks use Spark joins to verify that fact records map to valid dimension records.

Relationship Check	Purpose
fact_transaction.account_id → dim_account.account_id	Confirms transaction account references are valid
fact_transaction.customer_id → dim_customer.customer_id	Confirms transaction customer references are valid
fact_transaction.branch_id → dim_branch.branch_id	Confirms branch references are valid where branch ID is populated
fact_transaction.product_id → dim_product.product_id	Confirms product references are valid where product ID is populated
fact_customer_risk_score.customer_id → dim_customer.customer_id	Confirms risk score records map to valid customers
fact_daily_account_balance.account_id → dim_account.account_id	Confirms balance records map to valid accounts

Relationship Check	Purpose
fact_daily_account_balance.customer_id → dim_customer.customer_id	Confirms balance records map to valid customers

These checks detect orphan records and relationship issues in the curated warehouse layer.

11.11 Source-to-Target Reconciliation

The PySpark framework reconciles standardized/source-like datasets to curated models.

Reconciliation Check	Purpose
Standardized customers to dim_customer	Confirms customer row movement
Standardized accounts to dim_account	Confirms account row movement
Standardized branches to dim_branch	Confirms branch row movement
Standardized products to dim_product	Confirms product row movement
Standardized transactions to fact_transaction	Confirms transaction row movement
Standardized customer risk scores to fact_customer_risk_score	Confirms risk score row movement

These checks confirm that curated modeling did not unexpectedly lose records from the standardized layer.

11.12 Control-Total Validation

Control-total validation uses Spark aggregations to compare totals and grouped counts between standardized and curated datasets.

Control-Total Check	Purpose
Total transaction amount	Confirms financial totals remain consistent
Transaction counts by transaction type	Confirms grouped transaction distribution remains consistent
Account counts by account status	Confirms account grouping consistency
Other grouped totals where fields exist	Confirms business value consistency through transformation

Control totals are important because data movement can preserve row counts while still changing business values incorrectly.

11.13 Exception Records

The PySpark framework generates bounded exception-record outputs. These records are designed for data steward or engineer review when validation failures occur.

Exception records include fields such as:

Field	Purpose
Check category	Identifies the validation type
Check name	Identifies the failed rule
Dataset name	Identifies the affected dataset
Business key	Identifies the affected record
Failed column	Identifies the problematic column
Failed value	Shows the invalid or missing value
Error message	Explains the issue
Load date	Links the exception to a specific pipeline run

In the completed validation run, there were zero failed rules and zero exception records. The exception-record structure was still created so the framework can support future failed-record review.

11.14 PySpark DQ Output Design

The PySpark DQ framework writes both CSV and Parquet outputs.

Output Type	Purpose
Dataset profile results	Dataset-level profiling evidence
Column profile results	Column-level profiling evidence
Rule results	Validation rule outcomes
Business rule results	Accepted-value, range, referential-integrity, reconciliation, and control-total results
Exception records	Failed-record outputs for review
Summary CSV	Run-level summary evidence

Writing both CSV and Parquet supports local review and future scalable consumption.

11.15 PySpark DQ Validation Results

The PySpark DQ framework completed successfully.

Validation Area	Result
Single dim_customer validation	500 rows, 3 rules, 0 failures
Full curated dataset run	7 datasets validated successfully
Phase 7A total rules	21 rules, all passed
Accepted-value failed count	0
Range failed count	0
Referential-integrity failed count	0
Reconciliation failed count	0
Control-total failed count	0
Exception record count	0
Runtime output handling	Runtime outputs ignored by Git
Final working tree	Clean

The PySpark validation results show that all seven curated datasets passed the implemented Spark-based profiling and validation framework.

11.16 Difference Between SQL DQ and PySpark DQ

The project intentionally includes both SQL and PySpark data quality frameworks because they practice different validation styles.

Area	SQL DQ Framework	PySpark DQ Framework
Execution engine	DuckDB	PySpark
Validation style	SQL scripts	Spark DataFrame logic
Primary skill practiced	SQL validation and reconciliation	Scalable Spark-based profiling and validation
Data access	Temporary DuckDB views over Parquet	Spark reads Parquet datasets
Rule style	SQL queries	Filters, joins, grouping, and aggregations
Output	SQL result CSVs and summary logs	CSV and Parquet DQ outputs and summary logs

The SQL layer demonstrates strong SQL-based data quality and reconciliation skills. The PySpark layer demonstrates distributed-style validation using Spark operations. Together, they provide broader coverage for Data Quality Engineering and Data Governance interview preparation.

11.17 PySpark Data Quality Outcome

The PySpark DQ framework provides scalable validation evidence for the curated warehouse layer.

Outcome	Result
DQ configuration	Completed
PySpark DQ runner	Completed
Dataset-level profiling	Completed
Column-level profiling	Completed
Schema validation	Completed
Required-column validation	Completed
Duplicate logical-key checks	Completed
Accepted-value checks	Completed
Range checks	Completed
Referential-integrity checks	Completed
Source-to-target reconciliation	Completed
Control-total checks	Completed
Exception-record generation	Completed
CSV and Parquet outputs	Completed
Summary logging	Completed
Final validation result	All seven curated datasets passed

This framework connects Spark-based engineering work to governance outcomes such as measurable quality controls, audit-readiness, stewardship review, and trust in curated data assets.

---

12. Governance Framework

12.1 Governance Framework Overview

The governance framework connects the completed technical pipeline to enterprise data governance concepts. The goal is to show how data engineering outputs become governed assets with business meaning, ownership, quality expectations, lineage, and stewardship accountability.

The governance framework was created after the source, ingestion, standardization, curated modeling, SQL DQ, and PySpark DQ layers were completed. It does not change pipeline execution logic. Instead, it documents and governs the existing pipeline.

The governance framework covers:

Governance Area	Purpose
Governance policy	Defines operating rules and responsibilities
Business glossary	Defines business terms used in the banking pipeline
Critical Data Elements	Identifies and classifies important governed fields
Data catalog	Documents technical assets across the pipeline
Source-to-target mapping	Documents how source data maps to curated outputs
Lineage mapping	Documents asset-level and field-level lineage
Data quality rule catalog	Documents validation categories implemented in SQL and PySpark
Stewardship issue examples	Demonstrates issue tracking and remediation concepts

12.2 Governance Objectives

The governance framework was designed to meet the following objectives:

Objective	Implementation
Define business meaning	Business glossary terms for banking concepts
Identify critical data	Critical Data Element inventory
Assign accountability	Owner and steward definitions
Classify data	PII, Confidential, Internal, and Critical Data Element classifications
Document assets	Data catalog entries for source, raw, standardized, curated, and DQ layers
Connect data movement	Source-to-target mappings and lineage records
Connect quality to governance	DQ rule catalog tied to implemented SQL and PySpark checks
Simulate stewardship	Sample issue records with severity, impact, owner, and remediation plan

Objective	Implementation
Support change control	Governance policy defines expectations for changes

12.3 Governance Artifacts

The governance framework includes Markdown and YAML artifacts.

Artifact	File	Purpose
Governance README	governance/README.md	Explains the governance folder structure and artifact purpose
Governance policy	governance/governance_policy.md	Defines governance operating rules
Business glossary	governance/glossary/business_terms.yml	Defines banking business terms
Critical Data Elements	governance/metadata/critical_data_elements.yml	Documents 15 CDEs and related metadata
Data catalog	governance/catalog/data_assets.yml	Catalogs key data assets
Source-to-target mapping	governance/metadata/source_to_target_mapping.yml	Maps source fields to curated outputs
Lineage mapping	governance/lineage/lineage_mapping.yml	Documents asset-level and field-level lineage
Data quality rule catalog	governance/data_quality/dq_rules.yml	Catalogs DQ rule categories
Stewardship issues	governance/stewardship/data_quality_issues.yml	Provides sample issue tracking records

These files simulate the type of metadata and governance content that would normally be managed in tools such as Collibra or OpenMetadata.

12.4 Governance Policy

The governance policy defines how governance should operate for the project. It establishes expectations for data domains, ownership, stewardship, critical data, classifications, quality controls, lineage, issue handling, and change control.

Governance Policy Area	Description
Data domains	Defines major business domains such as Customer, Account, Transaction, Product, Branch, and Risk
Data ownership	Identifies accountable business or functional owners
Data stewardship	Defines steward responsibilities for review and issue triage
Data quality engineering	Defines technical validation and remediation responsibilities
Critical Data Elements	Defines how important fields are identified and governed
Data classifications	Defines classification categories such as PII, Confidential, Internal, and Critical Data Element
Data quality expectations	Connects governed fields to measurable DQ checks
Lineage expectations	Requires traceability from source to curated outputs
Stewardship issue handling	Defines how issues are reviewed, assigned, remediated, and closed
Change control	Requires governed changes to be reviewed and documented

The governance policy provides the operating model for the rest of the governance artifacts.

12.5 Governance Roles

The project uses governance roles that mirror enterprise data governance programs.

Role	Responsibility
Data Owner	Accountable for business meaning, usage, and approval of data assets
Data Steward	Reviews data definitions, CDEs, quality issues, and remediation status
Data Engineer	Builds and maintains ingestion, processing, and curated data pipelines
Data Quality Engineer	Defines and validates DQ rules, reconciliations, exceptions, and evidence
Metadata Administrator	Supports catalog, lineage, classifications, and OpenMetadata-style configuration
Data Governance Lead	Oversees governance standards, operating model, approvals, and escalation

These roles help demonstrate how technical data work connects to governance accountability.

12.6 Data Domains

The governance framework organizes data around banking domains.

Data Domain	Example Assets and Fields
Customer	customers, dim_customer, customer ID, customer number, email, date of birth
Account	accounts, dim_account, account ID, account number, account status, current balance
Transaction	transactions, fact_transaction, transaction ID, transaction amount, transaction date, transaction type
Product	products, dim_product, product ID, product code, product type
Branch	branches, dim_branch, branch ID, branch code
Risk	customer_risk_scores, fact_customer_risk_score, risk score, risk band
Data Quality	data_quality_rules, SQL DQ outputs, PySpark DQ outputs
Stewardship	stewardship_issues, sample issue records

Domain grouping helps connect business ownership and stewardship responsibilities to technical assets.

12.7 Data Classification

The governance framework uses classifications to identify sensitivity and importance.

Classification	Meaning
PII	Personally identifiable information such as email and date of birth
Confidential	Sensitive business or financial fields
Internal	Internal operational fields not intended for public use
Critical Data Element	Fields that are important for reporting, risk, quality, governance, or business operations

Classifications are especially important for fields such as customer identifiers, email, date of birth, account identifiers, transaction amount, transaction date, risk score, and risk band.

12.8 Governance Validation

The governance artifacts were validated for completeness and consistency.

Validation Area	Result
Governance foundation files	Created successfully

Validation Area	Result
Business glossary	Created
Critical Data Elements	15 CDEs defined
Data catalog	Created
Source-to-target mappings	Created
Lineage examples	Created
DQ rule categories	Approved categories referenced
Stewardship issue records	Created as sample records
YAML readability	Validated
Pipeline logic impact	No pipeline code changed
SQL validation impact	No SQL validation logic changed
PySpark DQ impact	No PySpark validation logic changed
Runtime output handling	Runtime folders not committed
Git working tree	Clean after completion

The governance framework was added without changing existing technical pipeline behavior.

12.9 Governance Framework Outcome

The governance framework adds business and operational meaning to the technical pipeline.

Outcome	Result
Governance policy	Completed
Business glossary	Completed
Critical Data Elements	Completed
Data catalog	Completed
Source-to-target mapping	Completed
Lineage mapping	Completed
Data quality rule catalog	Completed

Outcome	Result
Stewardship issue examples	Completed
Ownership and stewardship concepts	Completed
Data classification	Completed
Governance-to-DQ linkage	Completed
Pipeline logic unchanged	Confirmed

This framework shows that data governance is not only about documentation. It connects technical assets, business definitions, ownership, quality rules, lineage, and stewardship into one governed operating model.

---

13. Data Catalog and Business Glossary

13.1 Data Catalog and Glossary Overview

The data catalog and business glossary provide the business and technical vocabulary for the project. Together, they help users understand what data exists, what it means, who owns it, how it is classified, and how it connects to the broader banking pipeline.

The business glossary defines important banking terms in business-friendly language. The data catalog documents technical data assets across the source, raw, standardized, curated, and data quality result layers.

These artifacts are important because governed data platforms require both business understanding and technical discoverability.

13.2 Business Glossary Purpose

The business glossary standardizes the meaning of important banking terms. Without a glossary, different teams may interpret the same term differently.

For example, “Transaction Amount” may be interpreted as posted amount, authorized amount, settled amount, debit amount, or credit amount depending on the system. A glossary provides a controlled definition and links that definition to physical fields and curated assets.

13.3 Business Glossary Coverage

The glossary includes terms such as:

Business Term	Purpose
Customer	Defines the banking customer entity

Business Term	Purpose
Account	Defines a banking account owned by or associated with a customer
Branch	Defines a banking branch entity
Product	Defines a banking product such as account or service type
Transaction	Defines a financial activity record
Transaction Amount	Defines the financial value of a transaction
Transaction Date	Defines the transaction activity date
Transaction Type	Defines the category of transaction activity
Account Balance	Defines the balance associated with an account
Customer Risk Score	Defines the numeric score used for customer risk monitoring
Risk Band	Defines the risk classification derived from or associated with risk scoring
Critical Data Element	Defines a field that requires stronger governance and quality controls

The glossary connects business terms to technical fields, curated assets, classifications, and data quality expectations.

13.4 Data Catalog Purpose

The data catalog documents the technical assets created or used in the project. It provides a structured inventory of data assets across the pipeline.

The catalog answers questions such as:

Catalog Question	Example Answer
What data assets exist?	Source tables, raw files, standardized files, curated models, DQ outputs
What layer does the asset belong to?	Source, raw, standardized, curated, data quality
What is the asset used for?	Ingestion, processing, reporting, validation, governance
Who owns or stewards the asset?	Data owner and data steward assignments
How is the asset classified?	PII, Confidential, Internal, Critical Data Element
What downstream assets depend on it?	Curated models, DQ outputs, lineage records

13.5 Cataloged Asset Types

The data catalog covers the main assets in the project.

Asset Type	Examples
PostgreSQL source tables	customers, accounts, transactions, branches, products, customer_risk_scores
Governance control tables	data_quality_rules, stewardship_issues, source_to_target_mappings
Raw Parquet assets	Raw extracts under data/raw
Standardized Parquet assets	Standardized outputs under data/standardized
Curated warehouse models	dim_customer, dim_account, dim_branch, dim_product, fact_transaction, fact_customer_risk_score, fact_daily_account_balance
SQL DQ result assets	SQL validation outputs under data/dq_results/sql
PySpark DQ result assets	PySpark validation outputs under data/dq_results/pyspark

This catalog structure shows the full lifecycle from source system to governed curated outputs and validation evidence.

13.6 Catalog Metadata Elements

Each catalog asset can be described using governance metadata elements.

Metadata Element	Purpose
Asset name	Identifies the data asset
Asset layer	Identifies source, raw, standardized, curated, or DQ layer
Description	Explains the asset purpose
Business domain	Links the asset to Customer, Account, Transaction, Risk, or other domains
Owner	Identifies business accountability
Steward	Identifies governance review responsibility
Classification	Identifies sensitivity or importance
Related CDEs	Links asset to Critical Data Elements
Related DQ rules	Links asset to quality expectations

Metadata Element	Purpose
Upstream assets	Shows where the asset came from
Downstream assets	Shows where the asset is used

This structure mirrors the kind of metadata captured in enterprise catalog tools.

13.7 Glossary-to-Catalog Relationship

The business glossary and data catalog are related but not the same.

Area	Business Glossary	Data Catalog
Primary focus	Business meaning	Technical asset inventory
Example item	Transaction Amount	fact_transaction.transaction_amount
Main user	Business users, data stewards, governance teams	Data engineers, analysts, stewards, metadata teams
Key purpose	Define terms consistently	Discover and understand assets
Governance value	Standardizes interpretation	Provides asset-level traceability

Together, the glossary and catalog help users understand both what a term means and where it exists in the technical pipeline.

13.8 Governance Value of Catalog and Glossary

The catalog and glossary support governance in several ways.

Governance Need	Catalog / Glossary Support
Data discovery	Users can identify available source, curated, and DQ assets
Business understanding	Glossary terms define key banking concepts
Ownership	Catalog assets can be linked to owners and stewards
Classification	Sensitive and critical fields can be marked clearly
DQ linkage	Assets and fields can be tied to validation rules
Lineage	Cataloged assets can be connected through source-to-target and lineage mappings
Stewardship	Issues can reference affected assets and business terms

These artifacts make the pipeline more explainable and easier to govern.

13.9 Data Catalog and Glossary Outcome

The data catalog and business glossary provide the business and metadata foundation for the project.

Outcome	Result
Business glossary terms	Created
Catalog entries	Created
Source assets cataloged	Completed
Raw assets cataloged	Completed
Standardized assets cataloged	Completed
Curated models cataloged	Completed
DQ result assets cataloged	Completed
Owners and stewards represented	Completed
Classifications represented	Completed
Glossary-to-asset connection	Completed
Governance and OpenMetadata mapping support	Completed

The catalog and glossary turn the project from a technical pipeline into a governed data platform simulation.

---

14. Critical Data Elements

14.1 Critical Data Element Overview

Critical Data Elements, also known as CDEs, are fields that require stronger governance because they are important for reporting, risk analysis, compliance, business operations, reconciliation, or decision-making.

In this project, CDEs were identified across customer, account, transaction, and risk domains. Each CDE is mapped to business meaning, domain, classification, owner, steward, related technical fields, related assets, and relevant data quality rule categories.

The CDE inventory helps demonstrate how governance teams identify and control the most important data fields in a pipeline.

14.2 CDE Selection Criteria

Fields were selected as CDEs based on the following criteria:

Selection Criterion	Description
Business importance	Field is important for business reporting or operational use
Risk relevance	Field supports risk scoring, risk classification, or risk monitoring
Financial relevance	Field affects transaction or account-level financial interpretation
Identity relevance	Field identifies customers, accounts, or transactions
Governance sensitivity	Field requires ownership, stewardship, or classification
Data quality dependency	Field requires quality checks such as null, duplicate, accepted-value, range, or reconciliation validation
Lineage importance	Field must be traceable from source to curated output

This approach ensures that the CDE list is focused on fields that matter most.

14.3 Critical Data Elements Defined

The project defines 15 Critical Data Elements.

CDE	Domain	Governance Importance
customer_id	Customer	Primary customer identifier used for joins, lineage, and relationship validation
customer_number	Customer	Business-facing customer identifier
email	Customer	Customer contact field and PII
date_of_birth	Customer	Customer identity and demographic field; treated as sensitive
account_id	Account	Primary account identifier used for joins and transaction relationships
account_number	Account	Business-facing account identifier
account_status	Account	Controlled field used for account lifecycle and reporting
current_balance	Account	Financial balance field requiring quality and reasonableness checks
transaction_id	Transaction	Primary transaction identifier
transaction_reference	Transaction	Business-facing transaction reference

CDE	Domain	Governance Importance
transaction_amount	Transaction	Financial transaction value used for reporting, reconciliation, and control totals
transaction_date	Transaction	Transaction activity date used for reporting and lineage
transaction_type	Transaction	Controlled field used for transaction categorization
risk_score	Risk	Numeric customer risk score used for risk monitoring
risk_band	Risk	Risk classification used for segmentation and governance

These CDEs represent the most important governed fields in the project.

14.4 CDE Classification

CDEs are classified based on sensitivity and governance importance.

CDE	Classification Example
customer_id	Critical Data Element, Internal
customer_number	Critical Data Element, Confidential
email	Critical Data Element, PII
date_of_birth	Critical Data Element, PII
account_id	Critical Data Element, Internal
account_number	Critical Data Element, Confidential
account_status	Critical Data Element, Internal
current_balance	Critical Data Element, Confidential
transaction_id	Critical Data Element, Internal
transaction_reference	Critical Data Element, Confidential
transaction_amount	Critical Data Element, Confidential
transaction_date	Critical Data Element, Internal
transaction_type	Critical Data Element, Internal
risk_score	Critical Data Element, Confidential
risk_band	Critical Data Element, Confidential

This classification structure helps show which fields require stronger controls, ownership, and data quality monitoring.

14.5 CDE Mapping Across Layers

CDEs are mapped across source, standardized, and curated layers. This mapping supports traceability and lineage.

CDE	Source Layer	Standardized Layer	Curated Layer
customer_id	customers.customer_id	customers.customer_id	dim_customer.customer_id
customer_number	customers.customer_number	customers.customer_number	dim_customer.customer_number
email	customers.email	customers.email	dim_customer.email
date_of_birth	customers.date_of_birth	customers.date_of_birth	dim_customer.date_of_birth
account_id	accounts.account_id	accounts.account_id	dim_account.account_id, fact_transaction.account_id
account_number	accounts.account_number	accounts.account_number	dim_account.account_number
account_status	accounts.account_status	accounts.account_status	dim_account.account_status
current_balance	accounts.current_balance	accounts.current_balance	dim_account.current_balance
transaction_id	transactions.transaction_id	transactions.transaction_id	fact_transaction.transaction_id
transaction_reference	transactions.transaction_reference	transactions.transaction_reference	fact_transaction.transaction_reference
transaction_amount	transactions.transaction_amount	transactions.transaction_amount	fact_transaction.transaction_amount
transaction_date	transactions.transaction_date	transactions.transaction_date	fact_transaction.transaction_date
transaction_type	transactions.transaction_type	transactions.transaction_type	fact_transaction.transaction_type
risk_score	customer_risk_scores.risk_score	customer_risk_scores.risk_score	fact_customer_risk_score.risk_score

CDE	Source Layer	Standardized Layer	Curated Layer
risk_band	customer_risk_scores.risk_band	customer_risk_scores.risk_band	fact_customer_risk_score.risk_band

This mapping gives the project a clear CDE lineage structure from source to curated assets.

14.6 CDE-to-DQ Rule Linkage

Each CDE is connected to relevant data quality rule categories.

CDE Type	Relevant DQ Rules
Customer identifiers	Null checks, duplicate checks, referential-integrity checks, source-to-target reconciliation
Account identifiers	Null checks, duplicate checks, referential-integrity checks, source-to-target reconciliation
Transaction identifiers	Null checks, duplicate checks, source-to-target reconciliation
Financial amount fields	Null checks, range checks, control-total validation
Controlled categorical fields	Accepted-value checks
Risk score fields	Null checks, range checks
Risk band fields	Accepted-value checks
Date fields	Null checks and reasonableness checks where applicable
PII fields	Null checks, classification, ownership, and stewardship review

This connection between CDEs and DQ rules is important because governance should not stop at defining important fields. Important fields must also be measured and monitored.

14.7 CDE Ownership and Stewardship

Each CDE can be assigned governance accountability through owners and stewards.

Governance Role	CDE Responsibility
Data Owner	Accountable for business meaning and approval of the CDE
Data Steward	Reviews CDE definition, classification, quality issues, and remediation
Data Quality Engineer	Implements and validates quality rules for the CDE

Governance Role	CDE Responsibility
Data Engineer	Ensures the CDE is correctly processed across source, standardized, and curated layers
Data Governance Lead	Ensures CDE governance standards are followed

Ownership and stewardship make the CDE inventory actionable rather than just descriptive.

14.8 CDE Governance Value

The Critical Data Element inventory supports the project in several ways.

Governance Need	CDE Support
Field prioritization	Identifies the most important fields for governance
Data quality monitoring	Links fields to validation rules
Lineage	Maps fields from source to curated assets
Cataloging	Connects fields to cataloged datasets
Classification	Identifies sensitive and critical fields
Stewardship	Supports issue assignment and remediation
Interview explanation	Demonstrates practical governance thinking beyond ETL

The CDE inventory is one of the strongest governance components in the project because it connects business importance, technical fields, quality rules, and stewardship.

14.9 Critical Data Element Outcome

The CDE work completed the field-level governance foundation for the project.

Outcome	Result
CDE inventory	15 CDEs defined
Business meaning	Documented
Domain mapping	Completed
Classification	Completed
Owner and steward mapping	Completed
Source field mapping	Completed

Outcome	Result
Standardized field mapping	Completed
Curated field mapping	Completed
Related asset mapping	Completed
Related DQ rule mapping	Completed
Governance readiness	Completed

The CDE inventory demonstrates how a banking data pipeline can identify and govern the fields that matter most for trust, reporting, reconciliation, risk, and compliance-style review.

---

15. Data Lineage and Source-to-Target Mapping

15.1 Lineage and Mapping Overview

Data lineage and source-to-target mapping explain how data moves from the PostgreSQL source system through raw, standardized, and curated layers. They provide traceability for business users, data engineers, data stewards, data quality teams, and audit-style review.

The project includes both source-to-target mappings and lineage mappings.

Artifact	Purpose
Source-to-target mapping	Shows how source tables and fields map into curated warehouse models
Lineage mapping	Shows how assets and fields flow through the full pipeline
Data catalog linkage	Connects mapped assets to cataloged datasets
CDE linkage	Connects important fields to field-level lineage
DQ linkage	Connects governed assets to validation checks and DQ outputs

Together, these artifacts explain where data comes from, how it changes, where it lands, and how it is validated.

15.2 Source-to-Target Mapping Purpose

Source-to-target mapping documents how source data is transformed into curated outputs. This is critical in governed environments because downstream users need to understand how reporting fields are created.

The mapping answers questions such as:

Question	Example
Which source table feeds this curated model?	customers feeds dim_customer
Which source fields become target fields?	transactions.transaction_amount becomes fact_transaction.transaction_amount
Is the field renamed, standardized, or transformed?	Email is standardized to lowercase
Are joins required?	fact_transaction joins transactions to accounts
What is the business meaning of the field?	Transaction amount represents the financial value of transaction activity
Which DQ rules validate the mapped field?	Transaction amount is validated through null, range, reconciliation, and control-total checks

15.3 Major Source-to-Target Mapping Paths

The project maps the following major source-to-target paths.

Source Table(s)	Target Curated Model	Mapping Purpose
customers	dim_customer	Creates customer dimension
accounts	dim_account	Creates account dimension
branches	dim_branch	Creates branch dimension
products	dim_product	Creates product dimension
transactions, accounts	fact_transaction	Creates transaction fact with account, customer, product, and branch context
customer_risk_scores	fact_customer_risk_score	Creates customer risk score fact
accounts, transactions	fact_daily_account_balance	Creates account-day balance summary fact

These mappings support source-to-curated traceability across the main banking domains.

15.4 Example Field-Level Source-to-Target Mappings

Source Field	Standardized Field	Curated Field	Notes
customers.customer_id	customers.customer_id	dim_customer.customer_id	Customer identifier preserved
customers.customer_number	customers.customer_number	dim_customer.customer_number	Business customer identifier preserved
customers.email	customers.email	dim_customer.email	Email standardized to lowercase
accounts.account_id	accounts.account_id	dim_account.account_id	Account identifier preserved
accounts.account_number	accounts.account_number	dim_account.account_number	Business account identifier preserved
accounts.account_status	accounts.account_status	dim_account.account_status	Status normalized and validated
transactions.transaction_id	transactions.transaction_id	fact_transaction.transaction_id	Transaction identifier preserved
transactions.transaction_reference	transactions.transaction_reference	fact_transaction.transaction_reference	Business transaction reference preserved
transactions.transaction_amount	transactions.transaction_amount	fact_transaction.transaction_amount	Financial value validated

Source Field	Standardized Field	Curated Field	Notes
			through DQ checks
transactions.transaction_date	transactions.transaction_date	fact_transaction.transaction_date	Transaction date preserved for reporting
transactions.transaction_type	transactions.transaction_type	fact_transaction.transaction_type	Controlled field validated through accepted-value checks
customer_risk_scores.risk_score	customer_risk_scores.risk_score	fact_customer_risk_score.risk_score	Numeric risk field validated through range checks
customer_risk_scores.risk_band	customer_risk_scores.risk_band	fact_customer_risk_score.risk_band	Controlled risk classification validated through accepted-value checks

These mappings support field-level lineage and help explain how CDEs move through the pipeline.

15.5 Asset-Level Lineage

Asset-level lineage shows how datasets move across the architecture.

Lineage Stage	Asset Examples
Source layer	PostgreSQL tables such as customers, accounts, transactions, customer_risk_scores
Raw layer	data/raw/<table_name>/load_date=YYYY-MM-DD/<table_name>.parquet
Standardized layer	data/standardized/<table_name>/load_date=YYYY-MM-DD/<table_name>.parquet
Curated layer	dim_customer, dim_account, fact_transaction, fact_customer_risk_score, fact_daily_account_balance
SQL DQ layer	SQL validation result files and summary logs
PySpark DQ layer	PySpark profiling, rule result, and exception output files
Governance layer	Catalog, glossary, CDEs, lineage, source-to-target mapping, and DQ rule catalog
OpenMetadata design layer	Metadata ingestion templates and lineage mapping concepts

This asset-level lineage supports technical walkthroughs and governance discussions.

15.6 Field-Level Lineage Examples

Field-level lineage was documented for important governed fields.

Field	Lineage Summary
customer_id	PostgreSQL customers.customer_id → raw customers Parquet → standardized customers → dim_customer.customer_id and downstream fact relationships
account_id	PostgreSQL accounts.account_id → raw accounts Parquet → standardized accounts → dim_account.account_id and fact_transaction.account_id
transaction_id	PostgreSQL transactions.transaction_id → raw transactions Parquet → standardized transactions → fact_transaction.transaction_id
transaction_amount	PostgreSQL transactions.transaction_amount → raw transactions Parquet → standardized transactions → fact_transaction.transaction_amount → SQL and PySpark control-total validation
transaction_date	PostgreSQL transactions.transaction_date → raw transactions Parquet → standardized transactions → fact_transaction.transaction_date
risk_score	PostgreSQL customer_risk_scores.risk_score → raw risk score Parquet → standardized risk scores → fact_customer_risk_score.risk_score

Field	Lineage Summary
risk_band	PostgreSQL customer_risk_scores.risk_band → raw risk score Parquet → standardized risk scores → fact_customer_risk_score.risk_band

These examples demonstrate how field-level lineage can support CDE governance, audit questions, and impact analysis.

15.7 Lineage and Data Quality Connection

Lineage is connected to data quality because validation checks need to know where fields originate and where they are consumed.

DQ Area	Lineage Connection
Null checks	Validate CDEs after they move from source to curated layer
Duplicate checks	Validate logical keys after ingestion and curation
Referential-integrity checks	Validate relationships created through modeled lineage
Source-to-target reconciliation	Confirms mapped records arrived in target assets
Control totals	Confirms mapped financial values remain consistent
Exception records	Identify records and fields that failed validation
Stewardship issues	Link failed fields to business terms, assets, and owners

This connection is important because governance teams need to understand not only that a check failed, but also where the failed data came from and which downstream assets are affected.

15.8 Lineage and Catalog Connection

Lineage and cataloging work together. Catalog entries identify the assets, while lineage explains how those assets are connected.

Catalog Element	Lineage Relationship
Source table	Starting point of lineage
Raw Parquet asset	First landed data asset
Standardized Parquet asset	Cleaned and schema-normalized data asset
Curated model	Downstream analytics-ready data asset
DQ result asset	Validation evidence derived from curated or standardized assets
Business glossary term	Business meaning connected to fields and assets

Catalog Element	Lineage Relationship
Critical Data Element	Field-level governance object traced through the pipeline

This makes the data catalog more useful because users can discover not only what an asset is, but also where it came from and where it goes.

15.9 Lineage and OpenMetadata Connection

The lineage and source-to-target artifacts were also designed to support future OpenMetadata representation.

Governance Artifact	OpenMetadata Concept
Source-to-target mapping	Lineage relationship definition
Lineage mapping	OpenMetadata lineage graph concept
Data catalog	OpenMetadata assets and tables
Business glossary	OpenMetadata glossary terms
Critical Data Elements	OpenMetadata tags and classifications
Ownership and stewardship	OpenMetadata owner and team concepts
DQ rule catalog	Data quality metadata concepts

OpenMetadata ingestion was not executed in the completed project. However, the lineage artifacts provide the metadata foundation for representing lineage in a governance platform later.

15.10 Lineage and Source-to-Target Outcome

The lineage and source-to-target mapping work completed the traceability layer of the project.

Outcome	Result
Source-to-target mappings	Completed
Major source-to-curated paths	Documented
Asset-level lineage	Completed
Field-level lineage examples	Completed
CDE lineage	Completed
DQ linkage	Completed
Catalog linkage	Completed

Outcome	Result
OpenMetadata lineage concept mapping	Completed
Pipeline logic impact	No pipeline logic changed

This section demonstrates that the project does not stop at building data outputs. It also explains how data moves, how fields are transformed, how governed fields are traced, and how validation results connect back to source and curated assets.

16. OpenMetadata Integration

16.1 OpenMetadata Integration Overview

The OpenMetadata portion of the project demonstrates how the completed governance artifacts could be represented in an open-source metadata management and data catalog platform. OpenMetadata was used as an open-source counterpart for practicing Collibra-style governance concepts such as data cataloging, business glossary management, ownership, stewardship, classifications, lineage, and metadata ingestion design.

This part of the project was intentionally implemented as a documentation and configuration-design layer. It did not modify any technical pipeline logic and did not execute a live OpenMetadata deployment or metadata ingestion job.

The purpose of this section is to explain how the project connects manual governance artifacts to OpenMetadata concepts in a controlled and honest way.

16.2 OpenMetadata Scope

The OpenMetadata scope included integration design, metadata ingestion templates, and a local deployment runbook.

Area	Status	Description
OpenMetadata integration design	Completed	Documented how project governance artifacts map to OpenMetadata concepts
PostgreSQL metadata ingestion template	Completed	Created a safe template for future PostgreSQL metadata ingestion
Curated layer metadata template	Completed	Created a template for representing curated warehouse assets
Lineage ingestion template	Completed	Created a template aligned to source-to-target and lineage artifacts
Local OpenMetadata runbook	Completed	Documented future local deployment and validation steps

Area	Status	Description
Secrets handling	Completed	Used environment-variable placeholders and committed no secrets
OpenMetadata installation	Not executed	Installation was intentionally not performed
Runtime metadata ingestion	Not executed	No ingestion jobs were run
Production deployment	Not implemented	The work remained local and documentation-driven

This boundary is important. The project practices OpenMetadata integration concepts, but it does not claim that OpenMetadata was fully deployed or used to ingest metadata at runtime.

16.3 OpenMetadata Artifacts Created

The OpenMetadata integration layer includes the following files:

File	Purpose
openmetadata/README.md	Documents the purpose of OpenMetadata in the project and explains how governance artifacts map to OpenMetadata concepts
openmetadata/ingestion_configs/postgres_metadata_ingestion.yml	Provides a safe metadata ingestion template for the PostgreSQL banking source system
openmetadata/ingestion_configs/curated_layer_metadata_ingestion.yml	Provides a template for representing curated warehouse assets
openmetadata/ingestion_configs/lineage_ingestion.yml	Provides a template for lineage ingestion concepts
openmetadata/local_runbook.md	Documents future local deployment, validation, and troubleshooting steps

These artifacts prepare the project for a future OpenMetadata deployment while keeping the completed project stable.

16.4 Governance-to-OpenMetadata Mapping

The OpenMetadata design maps the manual governance artifacts to equivalent OpenMetadata concepts.

Project Governance Artifact	OpenMetadata Concept
governance/glossary/business_terms.yml	OpenMetadata Glossary
governance/catalog/data_assets.yml	OpenMetadata Data Assets and Tables
governance/metadata/critical_data_elements.yml	OpenMetadata Tags and Classifications
Governance owner and steward definitions	OpenMetadata Owners, Teams, and Responsibilities
governance/lineage/lineage_mapping.yml	OpenMetadata Lineage Graph
governance/metadata/source_to_target_mapping.yml	Lineage and source-to-target relationship metadata
Phase 6 SQL DQ validation	OpenMetadata Data Quality concepts
Phase 7 PySpark DQ validation	OpenMetadata Data Quality concepts
governance/stewardship/data_quality_issues.yml	Stewardship and issue-management context

This mapping shows how manual governance work can later be represented inside a metadata platform.

16.5 PostgreSQL Metadata Ingestion Template

The PostgreSQL metadata ingestion template was created to represent future metadata ingestion from the banking source system.

Template Area	Purpose
Source connection structure	Documents how the PostgreSQL source system would be registered
Database metadata concepts	Represents tables, columns, schema metadata, and technical structure
Environment variable usage	Avoids hardcoded secrets
Secure placeholder usage	Keeps credentials out of source control
Source asset registration	Supports future catalog visibility for PostgreSQL tables

This template was not executed. Its purpose is to show how a metadata ingestion configuration could be prepared safely.

16.6 Curated Layer Metadata Template

The curated layer metadata template was created to represent future catalog registration for curated warehouse assets.

Curated Asset Type	Example Assets
Dimensions	dim_customer, dim_account, dim_branch, dim_product
Facts	fact_transaction, fact_customer_risk_score, fact_daily_account_balance
Governed warehouse assets	Curated Parquet datasets used for reporting, validation, lineage, and CDE mapping

The template is documentation-oriented. It does not require the project to move curated files into a cloud storage platform or execute a runtime ingestion process.

16.7 Lineage Ingestion Template

The lineage ingestion template was created to represent how lineage could be registered in OpenMetadata later.

Lineage Source	Lineage Target
PostgreSQL source tables	Raw Parquet assets
Raw Parquet assets	Standardized Parquet assets
Standardized Parquet assets	Curated warehouse models
Curated warehouse models	SQL DQ result assets
Curated warehouse models	PySpark DQ result assets
Governance lineage artifacts	OpenMetadata lineage graph concepts

This template aligns with the source-to-target and lineage mapping artifacts created in the governance layer.

16.8 Local OpenMetadata Runbook

The local runbook documents how OpenMetadata could be deployed and validated in a future enhancement.

The runbook covers:

Runbook Area	Description
Prerequisites	Docker, OpenMetadata requirements, PostgreSQL connectivity, and environment variables
Deployment sequence	Start PostgreSQL, deploy OpenMetadata separately, configure metadata connections, execute ingestion, and validate results
Catalog validation	Confirm source and curated assets are visible

Runbook Area	Description
Glossary validation	Confirm business glossary terms are represented
Ownership validation	Confirm owners and stewards are assigned
Classification validation	Confirm tags and classifications are visible
Lineage validation	Confirm lineage relationships are visible
Troubleshooting	Connection failures, missing assets, ingestion issues, and lineage visibility issues

The runbook is a future execution guide, not proof that OpenMetadata was deployed.

16.9 Security Controls

The OpenMetadata templates follow safe configuration practices.

Security Control	Result
No committed passwords	Completed
No committed tokens	Completed
No committed secrets	Completed
Environment-variable placeholders	Used
No runtime files generated	Confirmed
No Docker changes	Confirmed
No pipeline logic changes	Confirmed

This supports secure configuration management and prevents sensitive values from being stored in the repository.

16.10 OpenMetadata Outcome

The OpenMetadata integration layer successfully connected the manual governance work to a practical metadata platform design.

Outcome	Result
OpenMetadata integration documentation	Completed
Governance artifact mapping	Completed

Outcome	Result
PostgreSQL ingestion template	Completed
Curated layer ingestion template	Completed
Lineage ingestion template	Completed
Local deployment runbook	Completed
Secure placeholder usage	Completed
Secrets committed	No
OpenMetadata installation executed	No
Runtime ingestion executed	No
Pipeline logic modified	No

This section demonstrates governance platform awareness while keeping the implementation boundary accurate.

---

17. Agile / SDLC Delivery Model

17.1 Agile / SDLC Overview

The Agile / SDLC layer shows how the project's governance, catalog, lineage, data quality, and OpenMetadata-related work could be planned, tracked, validated, released, and supported in an enterprise delivery environment.

In real organizations, governance work is not only technical. Teams must define requirements, write user stories, estimate work, assign owners, document acceptance criteria, validate results, capture test evidence, approve releases, and support issues after release.

This project simulated that delivery model using local Markdown and YAML artifacts instead of a live JIRA instance.

17.2 Agile / SDLC Artifacts

The project includes the following Agile and SDLC artifacts:

Artifact	File	Purpose
User stories	agile/user_stories.md	Captures governance, catalog, lineage, DQ, and OpenMetadata requirements
Sprint backlog	agile/sprint_backlog.yml	Organizes work into a four-sprint Scrum delivery model

Artifact	File	Purpose
Definition of Done	agile/definition_of_done.md	Defines completion criteria for governance deliverables
Test evidence	agile/test_evidence.md	Captures validation evidence from completed project components
Release notes	agile/release_notes.md	Summarizes delivered project capabilities

These files simulate enterprise Agile delivery without requiring an actual JIRA workspace.

17.3 Agile Epics

The user stories were organized across five major epics.

Epic	Purpose
Governance Epic	Covers governance policy, ownership, stewardship, and Critical Data Elements
Catalog Epic	Covers data asset cataloging and metadata documentation
Lineage Epic	Covers source-to-target mapping and lineage documentation
Data Quality Epic	Covers SQL DQ, PySpark DQ, validation evidence, and issue tracking
OpenMetadata Epic	Covers OpenMetadata design, templates, and metadata platform mapping

These epics align directly with the project's technical and governance capabilities.

17.4 User Stories

The Agile layer includes 19 user stories. Each story includes a story ID, epic, title, story points, user story statement, and acceptance criteria.

The user stories cover areas such as:

Story Area	Example Focus
Governance policy	Define operating rules and responsibilities
Business glossary	Define banking business terms
Critical Data Elements	Identify and document governed fields
Data catalog	Catalog source, raw, standardized, curated, and DQ result assets
Ownership and stewardship	Assign accountable roles
Source-to-target mapping	Document field and asset movement

Story Area	Example Focus
Lineage documentation	Trace data from PostgreSQL to curated and DQ layers
SQL Data Quality	Document validation checks and evidence
PySpark Data Quality	Document Spark-based profiling and validation
Data Quality evidence	Capture validation results
OpenMetadata planning	Map governance artifacts to OpenMetadata concepts
Metadata ingestion templates	Prepare future OpenMetadata ingestion configuration

The stories make the project explainable from a delivery perspective, not just a technical perspective.

17.5 Sprint Backlog

The sprint backlog organizes the completed work into a four-sprint delivery structure.

Sprint	Sprint Focus	Main Outcome
Sprint 1	Governance Foundation	Governance policy, business glossary, and Critical Data Elements
Sprint 2	Catalog and Lineage	Data catalog, source-to-target mapping, and lineage artifacts
Sprint 3	Data Quality	SQL and PySpark DQ validation evidence and DQ rule catalog
Sprint 4	OpenMetadata	OpenMetadata integration design, ingestion templates, and local runbook

The sprint backlog includes sprint goals, assigned stories, priorities, owners, statuses, and story points. All story statuses were marked as Done because the backlog represents work completed across the project.

17.6 Delivery Roles

The Agile simulation includes role assignments that mirror enterprise governance delivery teams.

Role	Delivery Responsibility
Data Governance Lead	Owns governance standards, review, approval, and escalation
Data Steward	Reviews business definitions, CDEs, classifications, and stewardship issues
Data Engineer	Builds ingestion, processing, and curated data layers
Data Quality Engineer	Implements and validates DQ rules, reconciliation, exceptions, and evidence

Role	Delivery Responsibility
Metadata Administrator	Supports catalog, lineage, OpenMetadata mapping, and metadata ingestion configuration

These roles show how responsibility is distributed across governance and engineering functions.

17.7 Definition of Done

The Definition of Done defines the criteria required before governance and data quality work can be considered complete.

Definition of Done Area	Requirement
Governance deliverables	Glossary, CDEs, catalog, lineage, DQ rules, stewardship records, and policy completed
Documentation	Deliverables documented in Markdown or YAML
Validation	SQL DQ, PySpark DQ, YAML readability, OpenMetadata templates, and Git status reviewed
Review	Simulated role-based review by engineering, DQ, stewardship, governance, and metadata roles
Approval	Phase completion, evidence documentation, and release notes completed
Scope guardrails	No unintended pipeline, SQL, PySpark, governance, Docker, or runtime changes

The Definition of Done makes delivery quality explicit.

17.8 Test Evidence

The test evidence document consolidates validation evidence from completed project components.

Evidence Area	Captured Result
SQL Data Quality	Full SQL suite completed with 54 checks and 0 failed checks
PySpark Data Quality	All seven curated datasets passed and exception record count was 0
Governance Artifacts	Business glossary, 15 CDEs, governance policy, catalog, mappings, lineage, DQ rules, and sample stewardship issues created
OpenMetadata	README, ingestion templates, local runbook, secure placeholders, no installation, no runtime ingestion

Evidence Area	Captured Result
Agile Planning	19 user stories, 4-sprint backlog, epics, owners, priorities, statuses, and story points documented

This evidence supports audit-readiness and interview explanation.

17.9 Release Notes

The release notes summarize the complete capabilities delivered by the project.

Capability Area	Delivered Capability
Data Engineering	PostgreSQL source system, synthetic data, Python ingestion, raw Parquet, PySpark standardization, curated warehouse
Data Quality	SQL DQ, PySpark DQ, null checks, duplicate checks, accepted values, range checks, RI checks, reconciliation, control totals, exception records
Governance	Business glossary, CDEs, governance policy, ownership, stewardship, classifications, and operating rules
Catalog	Manual data catalog, asset descriptions, owners, stewards, classifications, and OpenMetadata mapping
Lineage	Source-to-target mapping, asset-level lineage, field-level lineage examples, and OpenMetadata lineage concepts
OpenMetadata	Integration design, ingestion templates, local runbook, secure placeholders, and governance platform mapping
Agile / SDLC	User stories, epics, sprint backlog, Definition of Done, test evidence, release notes, and stewardship workflow

The release notes make the final project easier to present as a completed delivery.

17.10 Agile / SDLC Outcome

The Agile / SDLC layer completed the requirement-to-release view of the project.

Outcome	Result
User stories	19 created
Epics	Governance, Catalog, Lineage, Data Quality, and OpenMetadata
Sprint backlog	4 sprints documented
Story points	Included

Outcome	Result
Owners	Included
Priorities	Included
Statuses	Included
Acceptance criteria	Included
Definition of Done	Completed
Test evidence	Completed
Release notes	Completed
Stewardship workflow	Completed
Pipeline logic changed	No

This layer shows that the project was not only built technically but also framed as an enterprise-style governance delivery.

---

18. Validation and Testing Results

18.1 Validation Overview

Validation was performed throughout the project to confirm that each layer produced the expected outputs and did not introduce unintended changes. The validation approach included source row-count checks, ingestion verification, PySpark processing checks, curated model validation, SQL DQ validation, PySpark DQ validation, governance artifact review, OpenMetadata template review, and Agile/SDLC evidence review.

The project used validation evidence to support technical correctness, governance readiness, and audit-style documentation.

18.2 Source Data Validation

The PostgreSQL source model was validated after schema creation and synthetic data generation.

Validation Area	Result
PostgreSQL schema created	Completed
Expected table count	10 tables
Synthetic data loaded	Completed

Validation Area	Result
Foreign key relationships	Preserved through generator logic
Environment configuration	.env used locally and ignored by Git
Final source row-count validation	Completed

Final source table row counts:

Table	Row Count
reference_codes	72
branches	10
products	6
customers	500
accounts	990
transactions	5,000
customer_risk_scores	742
data_quality_rules	12
stewardship_issues	8
source_to_target_mappings	25

18.3 Python Ingestion Validation

The Python ingestion layer was validated by comparing extracted PostgreSQL row counts to Parquet read-back row counts.

Table	Extracted Row Count	Parquet Row Count	Status
branches	10	10	SUCCESS
products	6	6	SUCCESS
reference_codes	72	72	SUCCESS
customers	500	500	SUCCESS
accounts	990	990	SUCCESS
transactions	5,000	5,000	SUCCESS

Table	Extracted Row Count	Parquet Row Count	Status
customer_risk_scores	742	742	SUCCESS
data_quality_rules	12	12	SUCCESS
stewardship_issues	8	8	SUCCESS
source_to_target_mappings	25	25	SUCCESS

Additional ingestion validation included:

Validation Control	Result
Python compilation	Passed
PostgreSQL connectivity	Passed
Table whitelist validation	Passed
Invalid table rejection	Passed
Load-date format validation	Passed
Invalid load-date rejection	Passed
Parquet read-back verification	Passed
Runtime folder Git ignore behavior	Confirmed
Final Git status	Clean

18.4 PySpark Standardized Processing Validation

The PySpark standardized processing layer was validated by comparing raw row counts to standardized row counts.

Table	Raw Row Count	Standardized Row Count	Status
branches	10	10	SUCCESS
products	6	6	SUCCESS
reference_codes	72	72	SUCCESS
customers	500	500	SUCCESS
accounts	990	990	SUCCESS
transactions	5,000	5,000	SUCCESS

Table	Raw Row Count	Standardized Row Count	Status
customer_risk_scores	742	742	SUCCESS
data_quality_rules	12	12	SUCCESS
stewardship_issues	8	8	SUCCESS
source_to_target_mappings	25	25	SUCCESS

Additional Spark processing validation included:

Validation Control	Result
Python compilation	Passed
Spark job execution	Passed
Table whitelist validation	Passed
Load-date validation	Passed
Required-column checks	Passed
Type casting	Completed
Standardized Parquet output	Created
Summary CSV output	Created
Runtime folder Git ignore behavior	Confirmed
Final Git status	Clean

18.5 Curated Warehouse Validation

The curated warehouse layer was validated through row counts, duplicate logical key checks, null foreign key counts, and join-loss tracking.

Model	Rows	Duplicate Keys	Null FK Rows	Status
dim_customer	500	0	0	SUCCESS
dim_account	990	0	0	SUCCESS
dim_branch	10	0	0	SUCCESS
dim_product	6	0	0	SUCCESS
fact_transaction	5,000	0	4,495	SUCCESS

Model	Rows	Duplicate Keys	Null FK Rows	Status
fact_customer_risk_score	742	0	0	SUCCESS
fact_daily_account_balance	990	0	0	SUCCESS

The fact_transaction null foreign key count was informational because it included optional relationship fields. The transaction enrichment produced 5,000 rows with 0 records lost to join.

18.6 SQL Data Quality Validation

The SQL Data Quality Framework completed successfully.

SQL Validation Area	Result
Null checks	8 checks, 0 failed
Duplicate checks	7 checks, 0 failed
Accepted-value checks	9 checks, 0 failed
Referential-integrity checks	14 checks, 0 failed
Source-to-target reconciliation	9 checks, 0 failed
Control totals	7 checks, 0 failed
Exception records	0 rows
Full SQL suite	54 checks, 0 failed

The SQL validation results confirm that the curated warehouse passed all implemented SQL quality checks, reconciliation checks, control totals, and exception-record queries.

18.7 PySpark Data Quality Validation

The PySpark Data Quality Framework also completed successfully.

PySpark Validation Area	Result
Single dim_customer run	500 rows, 3 rules, 0 failures
Full curated dataset run	7 datasets validated successfully
Phase 7A total rules	21 rules, all passed
Accepted-value failed count	0
Range failed count	0
Referential-integrity failed count	0

PySpark Validation Area	Result
Reconciliation failed count	0
Control-total failed count	0
Exception record count	0
Runtime output Git ignore behavior	Confirmed
Final Git status	Clean

This confirms that all seven curated datasets passed the implemented PySpark profiling and validation checks.

18.8 Governance Artifact Validation

The governance artifacts were validated for completeness and readability.

Governance Validation Area	Result
Governance README	Created
Governance policy	Created
Business glossary	Created
Critical Data Elements	15 CDEs defined
Data catalog	Created
Source-to-target mappings	Created
Lineage mappings	Created
DQ rule catalog	Created
Sample stewardship issue records	Created
YAML readability	Validated
Pipeline logic changed	No
SQL validation logic changed	No
PySpark validation logic changed	No
Git status	Clean

18.9 OpenMetadata Validation

The OpenMetadata integration work was validated as documentation and template design.

OpenMetadata Validation Area	Result
OpenMetadata README	Created
PostgreSQL ingestion template	Created
Curated layer ingestion template	Created
Lineage ingestion template	Created
Local runbook	Created
Governance mappings	Documented
Environment-variable placeholders	Used
Secrets committed	No
Runtime files generated	No
OpenMetadata installation executed	No
Metadata ingestion executed	No
Pipeline logic changed	No
Git status	Clean

This confirms that the OpenMetadata work remained within its intended documentation and configuration-design scope.

18.10 Agile / SDLC Validation

The Agile / SDLC layer was validated as a documentation and delivery-evidence layer.

Agile / SDLC Validation Area	Result
User stories	19 created
Sprint backlog	4 sprints documented
Epics covered	Governance, Catalog, Lineage, Data Quality, OpenMetadata
Story points	Included
Owners	Included
Priorities	Included

Agile / SDLC Validation Area	Result
Status values	Included
Acceptance criteria	Included
Definition of Done	Created
Test evidence	Created
Release notes	Created
Stewardship workflow	Created
Pipeline files changed	No
Runtime validation executed in this phase	No
Git status	Clean

18.11 Consolidated Validation Summary

Project Area	Validation Result
Source data generation	10 tables loaded successfully
Python ingestion	All 10 tables extracted and Parquet row counts matched
PySpark standardization	All 10 tables processed and row counts matched
Curated modeling	All 7 curated models built successfully
SQL DQ	54 checks, 0 failed, 0 exception records
PySpark DQ	All 7 curated datasets passed, 0 exception records
Governance artifacts	Glossary, CDEs, catalog, lineage, DQ rules, policy, and stewardship examples completed
OpenMetadata	Documentation, ingestion templates, and runbook completed; no runtime ingestion
Agile / SDLC	19 stories, 4-sprint backlog, DoD, test evidence, release notes, and stewardship workflow completed

The validation results show that the project is technically complete, governance-ready, and suitable for portfolio and interview walkthroughs.

19. Governance Stewardship Workflow

19.1 Stewardship Workflow Overview

The governance stewardship workflow defines how data quality and governance issues are detected, triaged, assigned, remediated, validated, and closed.

This workflow is important because data governance does not end with cataloging, CDE definition, lineage mapping, or DQ rule execution. When issues are found, organizations need a clear operating process for ownership, investigation, remediation, validation, and closure.

The project defines the following stewardship workflow:

Detected → Triaged → Assigned → Remediated → Validated → Closed

19.2 Workflow Stages

Stage	Description
Detected	A governance or DQ issue is identified through SQL validation, PySpark validation, metadata review, catalog review, lineage review, or steward feedback
Triaged	The issue is reviewed for severity, business impact, affected asset, related CDE, and ownership
Assigned	The issue is assigned to the correct business or technical owner
Remediated	The assigned resolver fixes the source data, transformation logic, metadata artifact, documentation, rule definition, or operational process
Validated	The fix is confirmed through SQL checks, PySpark checks, metadata review, governance artifact review, or steward approval
Closed	The issue is closed after validation evidence is captured and the Data Steward or Governance Lead confirms completion

This workflow simulates a practical enterprise issue-management process for governed data assets.

19.3 Issue Detection Sources

Issues can be detected from multiple sources.

Detection Source	Example Issue
SQL DQ validation	Failed null check, duplicate key, control-total mismatch, or orphan record
PySpark DQ validation	Failed range check, accepted-value failure, or reconciliation mismatch

Detection Source	Example Issue
Metadata review	Missing owner, missing classification, or incomplete catalog entry
Glossary review	Ambiguous business definition
CDE review	Missing classification, missing steward, or missing DQ rule linkage
Lineage review	Missing source-to-target path or incomplete field lineage
Steward feedback	Business user reports incorrect or unclear data
Release review	Test evidence or approval criteria incomplete

This broad detection model reflects how data issues are found in real governance programs.

19.4 Issue Triage

Triage determines severity, ownership, and next steps.

Triage Attribute	Purpose
Severity	Determines urgency and escalation path
Business impact	Explains how the issue affects reporting, risk, reconciliation, or operations
Affected asset	Identifies the table, file, model, or artifact affected
Related CDE	Identifies whether a Critical Data Element is involved
Detected by rule	Links issue to a DQ rule or review process
Assigned steward	Identifies business review owner
Assigned resolver	Identifies technical or metadata resolver
Remediation plan	Defines corrective action
Validation method	Defines how the fix will be confirmed

Triage ensures that issues are handled based on risk and business impact rather than randomly.

19.5 Stewardship Roles

Role	Responsibility in Workflow
Data Steward	Reviews business impact, validates business meaning, and confirms closure
Data Quality Engineer	Investigates DQ rule failures, validates checks, and confirms DQ evidence
Data Engineer	Fixes ingestion, transformation, schema, mapping, or curated model issues

Role	Responsibility in Workflow
Data Owner	Provides accountability for business-critical decisions and escalations
Data Governance Lead	Oversees process adherence, prioritization, and governance approval
Platform Owner	Supports platform-level issues such as metadata tooling or runtime environment problems

These roles make issue ownership clear.

19.6 Issue Record Structure

Stewardship issue records include fields that support tracking and remediation.

Field	Purpose
issue_id	Unique issue identifier
title	Short issue description
status	Current workflow status
severity	Business or technical severity
related_asset	Affected data asset
related_cde	Affected Critical Data Element
detected_by_rule	DQ rule or review process that detected the issue
assigned_steward	Steward responsible for business review
business_impact	Explanation of downstream impact
remediation_plan	Planned fix
notes	Additional context

These fields support traceability from detection through closure.

19.7 Stewardship Workflow Outcome

The stewardship workflow adds operational governance to the project.

Outcome	Result
Issue lifecycle defined	Completed
Detection sources identified	Completed

Outcome	Result
Triage process defined	Completed
Assignment model defined	Completed
Remediation process defined	Completed
Validation process defined	Completed
Closure criteria defined	Completed
Escalation roles identified	Completed
Sample stewardship issue records created	Completed

This workflow demonstrates how data quality and governance issues would be managed after validation rules detect a problem.

---

20. JD Skill Alignment

20.1 JD Alignment Overview

The project was designed to align with a Data Governance, Collibra-style, Data Quality Data Engineer role. It covers data governance, data cataloging, data lineage, data quality, SQL, PySpark, Python scripting, enterprise governance tool concepts, JIRA-style Agile delivery, Scrum, SDLC, data modeling, data warehousing, and schema design.

The project does not claim Collibra implementation. It uses OpenMetadata as an open-source counterpart for practicing similar governance concepts.

20.2 JD-to-Project Mapping

JD Requirement	Project Implementation
Data Governance	Governance policy, business glossary, CDEs, classifications, owners, stewards, and stewardship issue examples
Data Catalog	Manual data asset catalog and OpenMetadata catalog mapping
Data Lineage	Source-to-target mapping, asset-level lineage, field-level lineage examples, and OpenMetadata lineage concept mapping
Data Quality	SQL DQ framework, PySpark DQ framework, DQ rule catalog, validation outputs, and exception-record structures

JD Requirement	Project Implementation
SQL	PostgreSQL schema, SQL validation scripts, DuckDB SQL execution, reconciliation, and control totals
PySpark	Raw-to-standardized processing, curated model creation, PySpark DQ profiling, Spark joins, aggregations, and validation
Python Scripting	Faker data generation, PostgreSQL ingestion, validation runners, logging, summary CSV generation, and automation support
Collibra / Informatica-style concepts	OpenMetadata used as an open-source counterpart for governance, catalog, glossary, ownership, classifications, and lineage concepts
JIRA	JIRA-style user stories and sprint backlog represented in Markdown/YAML
Agile	Sprint-based delivery simulation
Scrum	Epics, user stories, story points, sprint goals, backlog, and acceptance criteria
SDLC	Requirement, design, build, validation, evidence, release notes, and stewardship support
Data Modeling	Normalized source model and curated fact/dimension model design
Data Warehousing	Curated warehouse layer with facts and dimensions
Schema Design	PostgreSQL schema, table relationships, keys, constraints, and curated model specifications
Communication	Business glossary, governance policy, test evidence, release notes, and project documentation

20.3 Data Governance Skill Coverage

Governance Skill	Project Evidence
Governance policy writing	governance/governance_policy.md
Business glossary design	governance/glossary/business_terms.yml
Critical Data Element management	governance/metadata/critical_data_elements.yml
Ownership and stewardship	Governance roles, owners, stewards, and issue records
Data classification	PII, Confidential, Internal, and Critical Data Element classifications

Governance Skill	Project Evidence
Governance operating model	Policy, workflow, issue lifecycle, and SDLC artifacts
Change control	Definition of Done, scope guardrails, and Git discipline

20.4 Data Quality Skill Coverage

Data Quality Skill	Project Evidence
Null validation	SQL and PySpark DQ checks
Duplicate validation	Logical-key duplicate checks
Accepted-value validation	Controlled field checks
Range validation	Transaction amount, balance, and risk score checks
Referential integrity	SQL checks and Spark join-based validations
Source-to-target reconciliation	Standardized-to-curated comparisons
Control totals	Financial and count-based validations
Exception records	SQL and PySpark exception-record structures
Test evidence	Consolidated validation evidence in Agile/SDLC layer

20.5 Data Engineering Skill Coverage

Data Engineering Skill	Project Evidence
Source schema design	Normalized PostgreSQL banking source model
Synthetic data generation	Faker data generator
Python ingestion	PostgreSQL-to-raw Parquet extraction
Data lake structure	Raw, standardized, curated, and DQ result folders
PySpark processing	Raw-to-standardized transformation
Curated modeling	Seven fact/dimension-style models
Validation automation	Python SQL runner and PySpark DQ runner
Logging and evidence	Summary CSVs and runtime logs
Git discipline	Commits, tags, clean working tree checks

20.6 Metadata and Catalog Skill Coverage

Metadata / Catalog Skill	Project Evidence
Data cataloging	governance/catalog/data_assets.yml
Glossary-to-asset mapping	Business glossary linked to technical fields and assets
CDE-to-asset mapping	CDEs mapped to source, standardized, and curated fields
Ownership metadata	Owners and stewards represented
Classification metadata	PII, Confidential, Internal, and CDE classifications
OpenMetadata mapping	OpenMetadata README and ingestion templates
Lineage metadata	Source-to-target and lineage mapping files

20.7 Agile / SDLC Skill Coverage

Agile / SDLC Skill	Project Evidence
User story writing	19 user stories
Epic definition	Governance, Catalog, Lineage, DQ, OpenMetadata epics
Sprint planning	4-sprint backlog
Story-point estimation	Included in user stories and sprint backlog
Acceptance criteria	Included for every user story
Definition of Done	Created
Test evidence	Created
Release notes	Created
Stewardship workflow	Created
Requirement-to-release traceability	Demonstrated through Agile artifacts

20.8 Interview Readiness Summary

This project prepares a strong interview narrative because it connects implementation with governance outcomes.

A concise interview explanation is:

“I built a local Banking Data Governance and Data Quality Pipeline using PostgreSQL, Python, PySpark, SQL, DuckDB, Parquet, OpenMetadata concepts, and Agile/SDLC artifacts. I started with a normalized synthetic banking source model, ingested the data into raw Parquet using Python, standardized it with PySpark, and created seven curated fact and dimension models. Then I built two separate data quality

frameworks: one SQL-based using DuckDB over Parquet and one PySpark-based using Spark DataFrame logic. The SQL framework completed 54 checks with 0 failures, and the PySpark framework validated all seven curated datasets with 0 exception records. On top of the technical pipeline, I created governance artifacts including a business glossary, 15 Critical Data Elements, a data catalog, source-to-target mappings, lineage, DQ rule catalog, stewardship issues, and a governance policy. I also mapped those artifacts to OpenMetadata concepts through documentation, ingestion templates, and a local runbook. Finally, I created Agile/SDLC artifacts such as user stories, a sprint backlog, Definition of Done, test evidence, release notes, and a stewardship workflow."

This explanation is accurate, complete, and does not overstate the implementation.

---

21. Lessons Learned

21.1 Technical Lessons Learned

The project reinforced several technical data engineering lessons.

Lesson	Explanation
A strong source model matters	Governance and DQ work becomes more meaningful when the source model includes realistic relationships, business identifiers, reference data, and control tables
Raw ingestion should be validated	Extracting data is not enough; row-count verification, load-date validation, table whitelisting, and logging are needed
Standardization creates downstream stability	Consistent naming, type casting, required-column checks, and metadata columns make curated modeling easier
Curated models need clear grain	Fact and dimension models must define exactly what one row represents
Parquet does not enforce keys	Logical keys must be documented and validated through checks
SQL remains essential	SQL is still one of the strongest tools for data quality validation, reconciliation, and audit evidence
PySpark DQ adds scalable validation practice	Spark joins, filters, grouping, and aggregations are useful for large-scale profiling and validation
Reconciliation is more than row counts	Control totals are needed because row counts can pass while financial values are incorrect

21.2 Governance Lessons Learned

The project also reinforced governance lessons.

Lesson	Explanation
Governance is more than cataloging	A catalog is useful, but governance also requires definitions, CDEs, ownership, DQ rules, lineage, and stewardship
Business terms need technical mapping	Glossary terms become useful when linked to physical fields and assets
CDEs should be actionable	Critical fields should connect to classifications, owners, stewards, lineage, and DQ checks
Lineage improves trust	Users need to know where data came from, how it changed, and where it is consumed
Stewardship needs workflow	Data quality issues need detection, triage, assignment, remediation, validation, and closure
Governance tools need preparation	Platforms such as OpenMetadata or Collibra require well-structured metadata, mappings, ownership, and classifications

21.3 Delivery Lessons Learned

The Agile/SDLC portion of the project showed that governance work benefits from structured delivery.

Lesson	Explanation
User stories clarify governance requirements	Governance tasks become easier to execute when written as stories with acceptance criteria
Definition of Done prevents ambiguity	Completion criteria help avoid unfinished or unclear deliverables
Test evidence matters	Validation results should be documented, not only executed
Release notes improve communication	Stakeholders need a clear summary of what was delivered
Scope control protects quality	Keeping each phase focused prevented unnecessary changes and confusion
Git discipline supports traceability	Commits, tags, clean status checks, and controlled file changes support SDLC evidence

21.4 Personal Learning Outcome

The most important learning outcome from this project is the ability to explain how data engineering, data quality, metadata management, data governance, and Agile delivery connect in one pipeline.

The project moved beyond isolated coding exercises. It showed how a governed data platform requires:

Capability	Why It Matters
Source modeling	Defines trusted upstream structure
Ingestion	Moves data into the analytical environment
Processing	Standardizes and prepares data
Curated modeling	Creates analytics-ready assets
Data quality	Measures and validates trust
Governance	Defines meaning, ownership, classification, and accountability
Lineage	Explains movement and transformation
Cataloging	Makes assets discoverable
Stewardship	Handles issues and remediation
Agile/SDLC	Delivers the work in a controlled enterprise process

This is the end-to-end understanding expected from a strong Data Governance or Data Quality Data Engineer.

---

22. Future Enhancements

22.1 Future Enhancement Overview

The completed project is a strong local practice lab. Future enhancements can make it closer to an enterprise-style platform. These enhancements should be treated as future work, not as completed functionality.

22.2 OpenMetadata Runtime Deployment

A logical future enhancement is to deploy OpenMetadata locally and execute the metadata ingestion templates.

Enhancement	Purpose
Deploy OpenMetadata locally	Move from documentation-only integration to a working metadata platform
Register PostgreSQL source	Catalog the banking source tables
Register curated assets	Represent curated fact and dimension models
Load glossary terms	Add business glossary terms into OpenMetadata

Enhancement	Purpose
Add tags and classifications	Represent PII, Confidential, Internal, and CDE classifications
Assign owners and stewards	Represent governance accountability
Register lineage	Visualize source-to-curated lineage
Validate catalog output	Confirm assets, glossary, classifications, and lineage are visible

This would convert the current OpenMetadata design into an executed governance platform demonstration.

22.3 Orchestration

The project can be enhanced by adding an orchestration layer.

Enhancement	Purpose
Add Airflow or another scheduler	Orchestrate source load, ingestion, standardization, curated build, SQL DQ, and PySpark DQ
Add task dependencies	Ensure each layer runs only after upstream success
Add retries	Improve operational reliability
Add run parameters	Support load-date-driven execution
Add operational alerts	Notify on validation failures or pipeline errors

This would make the project closer to production-style pipeline operations.

22.4 CI/CD and Automated Testing

A future CI/CD layer could improve code quality and deployment discipline.

Enhancement	Purpose
Add GitHub Actions or Azure DevOps pipeline	Automate validation on pull requests
Run Python compilation checks	Catch syntax issues early
Run unit tests	Validate helper functions and configuration logic
Validate YAML files	Ensure governance and OpenMetadata files remain readable
Run selected DQ checks	Confirm validation logic is stable

Enhancement	Purpose
Enforce formatting	Improve maintainability
Generate release artifacts	Strengthen SDLC delivery evidence

This would strengthen the engineering maturity of the project.

22.5 Cloud or Object Storage Extension

The local Parquet structure can be extended to cloud-style object storage.

Enhancement	Purpose
Use MinIO locally	Simulate S3-style object storage
Extend to AWS S3, ADLS Gen2, or GCS	Practice cloud data lake architecture
Register external locations	Prepare for governance and catalog integration
Add partition management	Improve organized storage layout
Add lifecycle policies	Simulate storage management practices

This would make the data lake design more enterprise-relevant.

22.6 Data Quality Enhancements

The DQ frameworks can be expanded with more advanced rules and reporting.

Enhancement	Purpose
Add threshold-based rules	Support warning and failure thresholds
Add trend checks	Compare current load to previous load
Add anomaly detection	Identify unusual volume or amount changes
Add DQ scorecards	Summarize quality by asset and domain
Add rule severity model	Prioritize critical failures
Add historical DQ reporting	Track quality trends over time
Add automated stewardship issue creation	Convert failed rules into issue records automatically

These enhancements would make the DQ framework more operational.

22.7 Governance Automation

Governance artifacts can be generated or validated automatically.

Enhancement	Purpose
Extract schema metadata automatically	Generate catalog drafts from source and curated schemas
Generate CDE coverage reports	Check whether CDEs exist in expected assets
Validate glossary-to-field mappings	Ensure business terms are linked to physical fields
Validate lineage completeness	Detect missing lineage paths
Generate governance summary reports	Produce executive-level governance status
Create issue records from DQ failures	Connect DQ validation to stewardship workflow

This would connect technical metadata automation with governance operations.

22.8 Reporting and Dashboarding

The project can be extended with a dashboard layer.

Enhancement	Purpose
DQ dashboard	Show rule pass/fail counts and exceptions
Lineage dashboard	Summarize source-to-curated mappings
Governance dashboard	Show CDE coverage, owner coverage, and classification coverage
Stewardship dashboard	Track issue status, severity, owner, and closure
Release dashboard	Summarize validation evidence by release

This would help communicate project status to stakeholders.

22.9 Data Model Enhancements

The banking model can be expanded for richer analytics.

Enhancement	Purpose
Add true daily balance snapshots	Improve fact_daily_account_balance accuracy
Add card transactions	Expand transaction domain
Add loans and payments	Broaden banking product coverage
Add customer householding	Support customer relationship analytics
Add fraud alerts	Support fraud monitoring use cases
Add slowly changing dimensions	Practice Type 2 history

Enhancement	Purpose
Add regulatory reporting fields	Strengthen governance and compliance-style use cases

These enhancements would make the model more realistic.

22.10 Future Enhancement Summary

The strongest next step is to deploy OpenMetadata locally and execute metadata ingestion. After that, the most valuable improvements would be orchestration, CI/CD, governance automation, DQ scorecards, and dashboarding.

Priority	Enhancement
1	Execute local OpenMetadata deployment and metadata ingestion
2	Add orchestration for end-to-end pipeline execution
3	Add CI/CD validation for code, YAML, and DQ checks
4	Add governance automation and metadata extraction
5	Add DQ scorecards and stewardship issue automation
6	Add dashboards for DQ, governance, lineage, and stewardship
7	Extend the banking model with additional domains and SCD patterns

These enhancements would move the project from a complete local practice lab toward a more enterprise-style governed data platform.

---

23. Final Conclusion

The Banking Data Governance & Data Quality Pipeline successfully demonstrates an end-to-end local practice lab for data engineering, data quality, metadata management, and governance concepts in a banking context.

The project starts with a synthetic PostgreSQL banking source system and moves through Python ingestion, raw Parquet landing, PySpark standardization, curated warehouse modeling, SQL validation, PySpark validation, governance documentation, OpenMetadata integration design, and Agile/SDLC delivery artifacts.

The completed pipeline includes a realistic 10-table source model, validated raw ingestion, standardized PySpark outputs, seven curated warehouse models, SQL data quality checks using DuckDB, Spark-based data quality validation, governance artifacts, business glossary terms, 15 Critical Data Elements, catalog entries, source-to-target mappings, lineage documentation, data quality rule catalog, stewardship issue examples, OpenMetadata documentation and templates, and Agile delivery evidence.

The project's strongest technical outcome is that it does not stop at data movement. It validates trust through both SQL and PySpark. The SQL Data Quality Framework completed 54 checks with 0 failures and 0 exception records. The PySpark Data Quality Framework validated all seven curated datasets with 0 failed rule counts and 0 exception records. These results provide measurable evidence that the curated data layer passed the implemented quality controls.

The project's strongest governance outcome is that it connects technical assets to business meaning, ownership, classification, lineage, quality rules, and stewardship. The business glossary defines important banking concepts. The CDE inventory identifies the most important governed fields. The catalog documents technical assets. The source-to-target and lineage artifacts explain how data moves. The DQ rule catalog connects validation categories to governance. The stewardship workflow explains how issues would be handled after detection.

The OpenMetadata section demonstrates governance-platform awareness without overstating the implementation. OpenMetadata was used as an open-source counterpart for Collibra-style concepts. The project includes integration documentation, metadata ingestion templates, and a local runbook, but it does not claim that OpenMetadata was installed or that runtime ingestion was executed.

The Agile / SDLC layer strengthens the project by showing how governance work can be delivered in an enterprise-style process. The project includes user stories, epics, a sprint backlog, story points, owners, acceptance criteria, Definition of Done, test evidence, release notes, and a stewardship workflow. This demonstrates the full lifecycle from requirement to release and support.

Overall, this project is a complete and explainable portfolio asset for Data Governance, Data Quality Engineering, Data Catalog, Data Lineage, Metadata Management, SQL, PySpark, Python, data modeling, data warehousing, and Agile/SDLC preparation. It provides a strong foundation for interview discussions because every major component has a clear purpose, implemented output, validation result, and governance connection.

The final project can be summarized as:

A local banking data governance and data quality platform that generates synthetic source data, ingests it into raw Parquet, standardizes it with PySpark, builds curated warehouse models, validates data quality through SQL and PySpark, documents governance artifacts, maps metadata concepts to OpenMetadata, and simulates Agile/SDLC delivery with audit-ready evidence.